

# 세션 4 · 장애대응 · 모니터링

운영 절차서 · 런북 · 장애대응 · 모니터링

---

## 대상 버전

Redis 8.10.x · MongoDB 8.3.x · Kafka 4.3.x

## OS

Ubuntu 24.04 LTS

## 실습 환경

Azure

## 관측 도구

Prometheus · Grafana

# 01

## 인프라 구축과 관측 스택

단계별 스택

ops 노드

exporter

Prometheus

Grafana

평상시 값

# 이 세션이 답하는 질문

---

- **앞 세 세션:** 무엇을 정지시킬지 알려 주고 시작. 예상한 상태로 바뀌었는지 확인
- **이 세션:** 증상만 주고 시작. 지표로 원인을 좁혀 되돌림
- **다루는 운영:** 탐지 · 감별 · 완화 · 복구 · 예방
- **다루지 않는 것:** 설치와 구성 절차를 새로 배우지 않음. 앞 세 세션의 플레이북을 그대로 돌림

**원인을 모르는 상태에서 시작합니다.**

앞 세 세션에서는 수강생이 직접 만든 상황이라 원인을 알고 있었습니다. 여기서는 화면에 보이는 증상만 주어지고, 무엇이 잘못됐는지는 지표로 좁혀 가야 합니다.

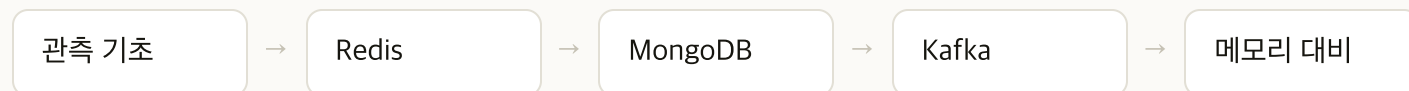
# 이 세션의 산출물

| 산출물                           | 사용 시점               | 답하는 질문    |
|-------------------------------|---------------------|-----------|
| <b>RB-OBS</b> 기본 모니터링         | 평상시. 당번이 아침에, 교대할 때 | 지금 정상인가   |
| <b>RB-R01</b> Redis 복제 이탈     | 증상을 만났을 때           | 어떻게 되돌리는가 |
| <b>RB-M01</b> MongoDB 복제 지연   | 증상을 만났을 때           | 어떻게 되돌리는가 |
| <b>RB-K01</b> Consumer Lag 급증 | 증상을 만났을 때           | 어떻게 되돌리는가 |
| <b>RB-R02</b> 메모리 압박          | 증상을 만났을 때           | 어떻게 되돌리는가 |

**RB-OBS** 가 나머지 넷을 가리킵니다.

평상시 점검에서 지표가 정상 범위를 벗어나면 그 줄이 어느 런북으로 갈지 알려 줍니다. 이 연결이 없으면 지표를 보고도 다음 행동으로 이어지지 않습니다.

# 세션 진행 순서



- **서비스 하나를 끝까지:** 지표 · 대시보드 · 알람 · 대표 장애 · 재현 · 런북
- **그다음 서비스로:** 같은 형태가 세 번 반복됨
- **앞에 모으는 것:** 관측의 기초와 런북의 형식. 서비스에 종속되지 않는 내용

세 번째 서비스에서는 다음에 무엇이 올지 알고 따라옵니다.

구간의 형태가 같아서 첫 구간에서 익힌 순서가 그대로 반복됩니다. 뒤로 갈수록 그 서비스에만 있는 것에 집중하게 됩니다.

# 단계별 스택

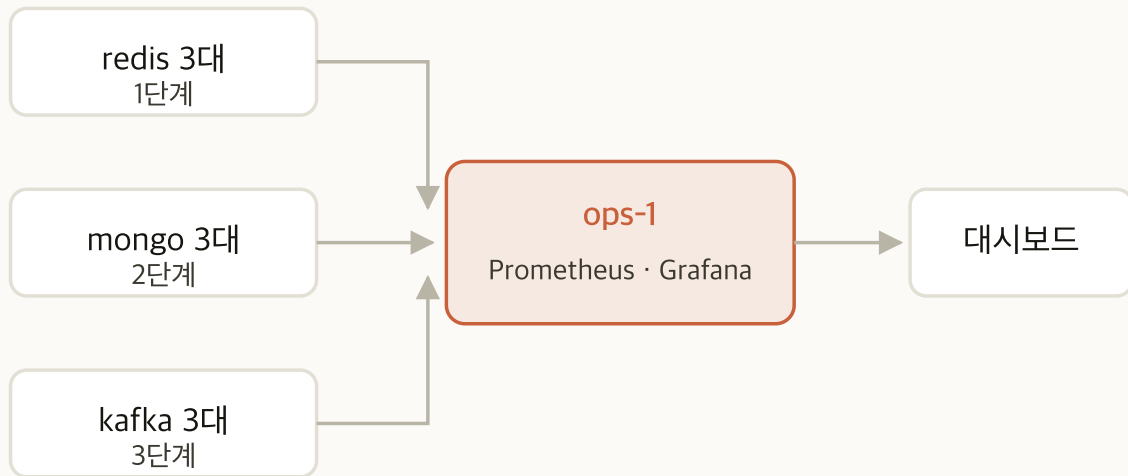
| 단계 | 구간               | 노드                              |
|----|------------------|---------------------------------|
| 1  | 관측 기초부터 Redis 까지 | ops-1 + redis-1·redis-2·redis-3 |
| 2  | MongoDB          | ops-1 + mongo-1·mongo-2·mongo-3 |
| 3  | Kafka            | ops-1 + kafka-1·kafka-2·kafka-3 |
| 4  | 메모리 대비           | ops-1 만                         |

- 유지되는 것: 네트워크, 보안 그룹, ops 노드. 세션 내내 그대로
- 갈아 끼우는 것: 서비스 노드 3대. `session.tfvars` 의 `stage` 를 고치고 `apply`

세 서비스가 동시에 있을 이유가 없습니다.

셋은 서로 독립적으로 돌고 지표도 장애도 섞이지 않습니다. 쓰지 않는 노드는 자원만 차지합니다.

# ops 노드 분리 근거



- **ops 노드가 하는 일:** Ansible 실행, 지표 수집, 부하 발생, 장애 주입, 런북 작성
- **서비스 노드를 겸하지 않음:** 관측 도구가 관측 대상과 함께 죽으면 실습이 성립하지 않음
- **부하 발생기도 여기:** 그래서 이 노드의 자원도 관측 대상

**관측 도구가 죽으면 아무것도 보이지 않습니다.**

Redis 노드에 Prometheus 를 함께 올렸다면 Redis 장애를 재현하는 순간 그 장애를 볼 수단이 같이 사라집니다. 장애 실습에서 이 분리는 선택이 아닙니다.

# 서비스별 사양 차이

| 서비스     | 계열      | 디스크     | 그 사양인 이유                          |
|---------|---------|---------|-----------------------------------|
| Redis   | 메모리 최적화 | 64 GiB  | 데이터 전체가 메모리에 있고 fork 여유가 필요       |
| MongoDB | 메모리 최적화 | 128 GiB | WiredTiger 캐시가 성능을 좌우             |
| Kafka   | 범용      | 256 GiB | 힙은 작고 page cache 가 주역. 쌓이는 것은 디스크 |
| ops     | 범용      | 64 GiB  | 수집과 부하 발생. 데이터를 쌓지 않음             |

같은 「메모리 부족」이 세 서비스에서 다르게 나타납니다.

사양이 다른 것이 그 차이의 첫 신호입니다. 무엇이 어떻게 다른지는 챕터 12에서 세 서비스를 나란히 놓고 다룹니다.



# 1단계 스택 생성

- 1 각자 PC 에서 `lab/terraform/04-operation` 으로 이동
- 2 `cp session.tfvars.example session.tfvars` 후 구독 ID 와 키 경  
로 입력
- 3 `terraform init`
- 4 `terraform plan -var-file=session.tfvars` 로 VM 4대 확인
- 5 `terraform apply -var-file=session.tfvars`
- 6 `ops-1` 공인 IP 로 SSH 접속

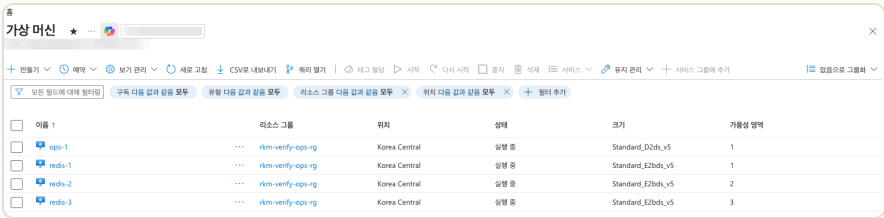
04-operation/main.tf · `stage` 가 노드를 정합합니다. mongo·kafka 도 같은 형태입니다

```
redis_nodes = var.stage == 1 ? {
  "redis-1" = { zone = "1", ip = "10.0.1.5" }
  "redis-2" = { zone = "2", ip = "10.0.1.6" }
  "redis-3" = { zone = "3", ip = "10.0.1.7" }
} : {}
```

`-var-file=session.tfvars` 를 빠뜨리면 변수 입력 프롬프트에서 멈춥니다.

`session.tfvars` 는 Terraform 이 자동으로 읽는 이름이 아닙니다. `plan` · `apply` · `destroy` 모두에 붙입니다.

# 포털에서 배치 확인



VM 4대 목록. 노드 이름과 가용성 영역

| 확인 대상 | 포털 메뉴 | 확인 항목                                   |
|-------|-------|-----------------------------------------|
| VM 목록 | 가상 머신 | 4대. ops-1 과 redis-1 · redis-2 · redis-3 |
| 영역 배치 | 가상 머신 | Redis 노드가 영역 1·2·3에 하나씩                 |
| VM 크기 | 가상 머신 | Redis 는 E 계열, ops 는 D 계열                |
| 디스크   | 디스크   | Redis 64 GiB, ops 64 GiB                |

# 이 세션의 구성 작업

- **하는 것:** 그 단계의 서비스와 관측 스택을 플레이북 한 번으로 올림
- **바뀌는 것:** 없음. 설정값이 앞 세 세션과 같음
- **세 번 반복됨:** 단계가 바뀔 때마다 같은 플레이북을 다시 돌림
- **새로 보는 것:** 인벤토리가 바뀌면 수집 대상도 따라 바뀌는 구조

**구성이 익숙하므로 시간이 남습니다.**

그 시간을 지표와 장애에 씁니다. 다만 단계 전환이 두 번 있어 플레이북을 세 번 돌립니다.

# 이 단계의 exporter

| exporter                    | 어디에              | 제공 지표                  |
|-----------------------------|------------------|------------------------|
| <code>node_exporter</code>  | 4대 전부. ops 노드 포함 | CPU · 메모리 · 디스크 · 네트워크 |
| <code>redis_exporter</code> | Redis 노드 3대      | <code>INFO</code> 의 필드 |

- `node_exporter` 가 ops 노드에도: 부하를 주는 쪽이 먼저 한계에 닿는 상황을 가려내기 위함
- 단계가 바뀌면: 서비스 exporter 가 그 서비스 것으로 바뀜
- Kafka 단계에서 하나 더: JMX exporter 가 Broker JVM 안에 붙음

exporter 는 그 단계에 있는 노드에만 깔립니다.

플레이북이 인벤토리를 보고 대상을 정하므로, 없는 서비스의 exporter 를 깔려고 하지 않습니다.

# 플레이북 실행

- 1 각자 PC 에서 `terraform output -raw inventory > hosts.ini`
- 2 `scp -i ~/.ssh/rkm hosts.ini azureuser@<ops 공인 IP>:~/ansible/inventory/hosts.ini`
- 3 ops 노드에서 `~/ansible` 로 이동
- 4 `inventory/hosts.ini` 에 `[redis]` 와 `[ops]` 두 그룹이 있는 것 확인
- 5 `ansible -m ping all` 로 4대 응답 확인
- 6 `ansible-playbook play-operation.yml` 실행
- 7 마지막 요약에서 실패 노드가 없는 것 확인

```
- name: 관측 스택 기동
  hosts: ops
  become: true
  roles:
    - role: monitoring
      monitoring_target: server
      monitoring_scrape_ops: true
      # 기본값은 꺼짐. 앞 세션 화면 유지
      monitoring_node_dashboard: true
```

**인벤토리는 부팅할 때 만들어지지 않습니다.**

저장소에 들어 있는 것은 세션 1 의 예시 파일입니다. 1단계에서, 그리고 단계가 바뀔 때마다 Terraform output 을 받아 덮어써야 합니다.

# Prometheus 수집 대상 확인

**Targets**

All Unhealthy Collapse All

**node (4/4 up)** [Show less](#)

| Endpoint                                                                  | State | Labels                                                                                     | Last Scrape | Scrape Duration | Error |
|---------------------------------------------------------------------------|-------|--------------------------------------------------------------------------------------------|-------------|-----------------|-------|
| <a href="http://10.0.1.5:9100/metrics">http://10.0.1.5:9100/metrics</a>   | UP    | <code>instance="10.0.1.5:9100"</code> <code>job="node"</code> <code>service="redis"</code> | 5.309s ago  | 78.25ms         |       |
| <a href="http://10.0.1.6:9100/metrics">http://10.0.1.6:9100/metrics</a>   | UP    | <code>instance="10.0.1.6:9100"</code> <code>job="node"</code> <code>service="redis"</code> | 2.794s ago  | 70.26ms         |       |
| <a href="http://10.0.1.7:9100/metrics">http://10.0.1.7:9100/metrics</a>   | UP    | <code>instance="10.0.1.7:9100"</code> <code>job="node"</code> <code>service="redis"</code> | 2.758s ago  | 65.73ms         |       |
| <a href="http://127.0.0.1:9100/metrics">http://127.0.0.1:9100/metrics</a> | UP    | <code>instance="127.0.0.1:9100"</code> <code>job="node"</code> <code>service="ops"</code>  | 12.592s ago | 73.26ms         |       |

**redis (3/3 up)** [Show less](#)

| Endpoint                                                                | State | Labels                                                                                      | Last Scrape | Scrape Duration | Error |
|-------------------------------------------------------------------------|-------|---------------------------------------------------------------------------------------------|-------------|-----------------|-------|
| <a href="http://10.0.1.5:9121/metrics">http://10.0.1.5:9121/metrics</a> | UP    | <code>instance="10.0.1.5:9121"</code> <code>job="redis"</code> <code>service="redis"</code> | 8.627s ago  | 8.823ms         |       |

Targets 화면. 두 job 이 전부 UP

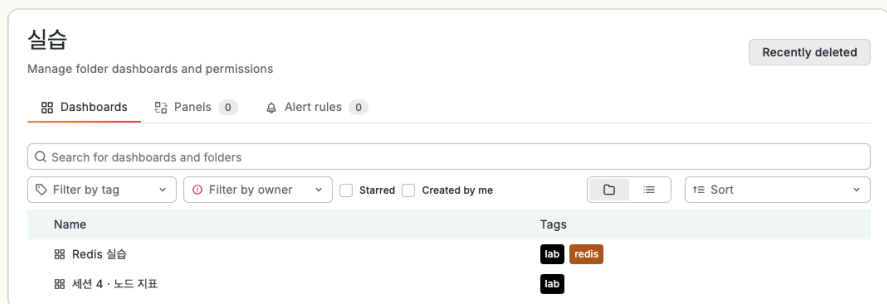
- 1 브라우저에서 ops 노드 공인 IP 의 **9090** 포트 접속
- 2 **Status** 에서 **Targets** 선택
- 3 job 둘과 대상 수 확인

| job          | 대상 수 | 수집 대상                 |
|--------------|------|-----------------------|
| <b>node</b>  | 4    | Redis 노드 3대 + ops 노드  |
| <b>redis</b> | 3    | <b>redis_exporter</b> |

**node** job 이 3대면 ops 노드에 **node\_exporter** 가 설치되지 않은 것입니다.

이 노드에서 부하를 발생시키므로 자원 지표가 있어야 합니다.

# Grafana 접속과 대시보드



대시보드 목록, 노드와 Redis

1 ops 노드 공인 IP 의 **3000** 포트 접속

2 초기 계정으로 로그인

3 대시보드 목록에서 둘을 확인

■ **노드**: CPU · 메모리 · 디스크 · 네트워크. 네 대 전부

■ **Redis**: Redis 의 기본 지표

■ **단계가 바뀌면**: 그 서비스의 대시보드가 올라옴

값이 그려지지 않으면 그 서비스의 **exporter** 를 먼저 봅니다.

대시보드는 있는데 패널이 비어 있으면 대시보드 문제가 아니라 수집 문제입니다. 앞 장의 Targets 화면으로 돌아갑니다.

# 수집 대상을 정하는 인벤토리

prometheus.yml.j2 · 인벤토리가 job 을 정합니다

```
scrape_configs:
{% if monitoring_present | default([]) %}
  - job_name: node
    static_configs:
{% for svc in monitoring_present %}
      - targets:
```

- **job\_name: node** 는 하나: 그 단계의 서비스마다 **static\_configs** 를 두고 **service** label 로 나눔
- **서비스별 job**: 인벤토리에 그 그룹이 있을 때만 생김
- **kafka\_jmx**: Kafka 그룹이 있을 때만 생김

인벤토리에 없는 그룹은 job 으로 나오지 않습니다.  
단계가 바뀌면 인벤토리가 바뀌고 수집 대상도 따라 바뀝니다.

단계를 바꾸면 인벤토리를 반드시 갱신합니다.

ops 노드의 인벤토리는 Terraform **output** 을 받아 덮어쓴 것입니다. 단계가 바뀌면 다시 받아 옮깁니다. 갱신하지 않으면 플레이북이 사라진 노드에 접속하려 다 그 자리에서 멈춥니다.



# Redis 기동 확인

- 1 `ansible redis -m shell -a "redis-cli ping"` 으로 3대 응답 확인
- 2 `ansible redis -m shell -a "redis-cli -p 26379 sentinel master mymaster"` 로 Sentinel 확인
- 3 `redis-cli -h <master 사설 IP> INFO replication` 으로 `connected_slaves` 확인

1번 명령의 출력

```
redis-1 | CHANGED | rc=0 >>
PONG
```

## 완료 판정

3대가 응답하고 `connected_slaves` 가 2이면 완료입니다.

# 앞 세 세션과 같은 구성

| 서비스     | 구성                               | 정지 시 반응        |
|---------|----------------------------------|----------------|
| Redis   | master 1 + replica 2, Sentinel 3 | Sentinel 자동 전환 |
| MongoDB | 3멤버 Replica Set, Arbiter 없음      | PRIMARY 선출     |
| Kafka   | Broker 3대, KRaft combined        | ISR 축소         |

- **바꾸지 않음**: 설정값이 앞 세 세션과 동일
- **쓰는 값**: 앞 세 세션에서 기록해 둔 설정값 그대로

앞 세 세션에서 남긴 기록을 꺼내 둡니다.

설정값과 명령 출력이 런북의 재료가 됩니다. 기록이 없으면 챕터 07 · 09 · 11 에서 쓸 것이 없습니다.

# 정상시 값 확인



두 대시보드의 정상 상태

- **지금 값을 적어 둬:** 장애가 나면 이 값과 비교
- **보는 것:** 노드 CPU·메모리·디스크 여유, Redis 의 기본 지표
- **아직 하지 않는 것:** 임계값 판단, 여러 지표를 조합한 원인 좁히기

**정상시 값을 모르면 이상한 값도 모릅니다.**

「CPU 40%」가 높은 것인지 낮은 것인지는 정상시 값을 알아야 판단할 수 있습니다. 여기서 적어 둔 값이 **RB-0BS** 의 정상 범위와 알람 임계값의 근거가 됩니다.

# 02

## 관측성 기초

지표와 로그

인프라 계층

서비스 계층

USE

RED

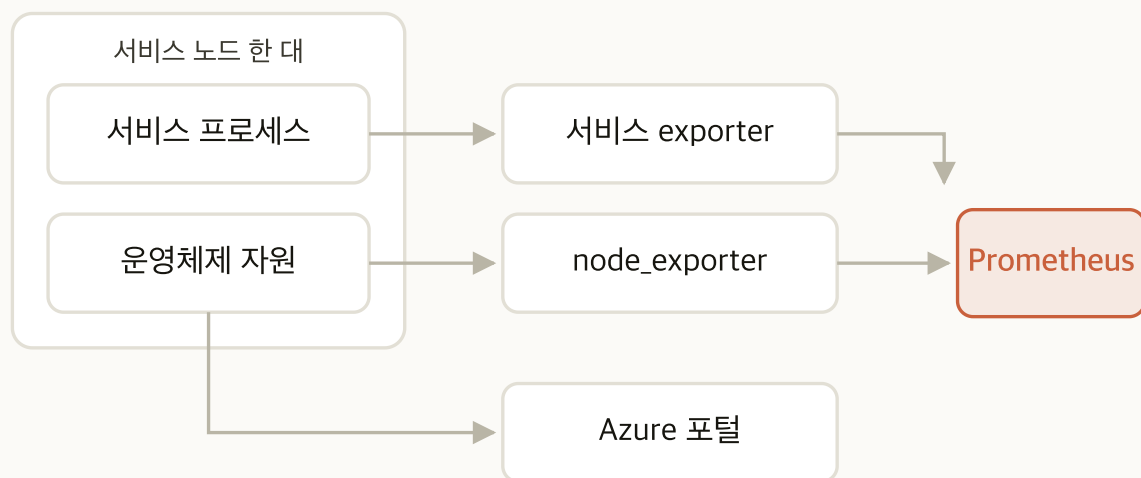
카운터와 게이지

# 지표와 로그

|          | 지표                   | 로그           |
|----------|----------------------|--------------|
| 답하는 질문   | 무엇이 이상한가             | 왜 그런가        |
| 형태       | 시각별 숫자               | 사건별 문장       |
| 강점       | 추세와 비교, 알람           | 원인과 맥락       |
| 약점       | 개별 사건의 사정            | 추세 판단, 알람    |
| 이 세션의 도구 | Prometheus · Grafana | 각 서비스의 로그 파일 |

런북 3절 「즉시 확인」의 절반이 로그입니다.  
지표가 「무엇이」를 줬혀 주면 그다음은 로그에서 「왜」를 찾습니다. 지표만 보고 대응하면 같은 증상의 다른 원인을 놓칩니다.

# 관측의 두 계층



| 계층  | 도구                    | 관측 대상                  |
|-----|-----------------------|------------------------|
| 인프라 | 포털 ·<br>node_exporter | CPU · 메모리 · 디스크 · 네트워크 |
| 서비스 | Prometheus · Grafana  | 각 서비스가 내는 지표           |

같은 노드에서 지표가 두 갈래로 나옵니다.

한쪽만 보면 반쪽만 보입니다. 두 계층을 함께 놓고 보는 것이 이 세션의 감별 방식입니다.

# 단일 계층 관측의 사각지대

| 상황              | 인프라 계층만 보면               | 서비스 계층의 관측 결과                      |
|-----------------|--------------------------|------------------------------------|
| 디스크 IOPS 가 급증   | 한계인 것은 알지만 <b>왜인지 모름</b> | 보이지 않음                             |
| 인덱스 없는 질의       | 보이지 않음                   | scanned 대 returned 비로 <b>바로 확정</b> |
| 디스크 공간 고갈       | 여유 지표로 <b>바로 확정</b>      | 보이지 않음                             |
| 프로세스가 OOM 으로 죽음 | 커널 로그에 남음                | 지표가 <b>끊길 뿐 이유가 남지 않음</b>          |

**Kafka 장애 원인 1순위인 디스크 고갈은 Kafka 지표에 나오지 않습니다.**

서비스 계층 지표만 보고 있으면 Broker 가 멈추는 순간까지 아무 신호도 못 봅니다. 노드 지표를 함께 봐야 하는 이유가 이것입니다.

# USE 방법론

| 항목            | 뜻          | 예                |
|---------------|------------|------------------|
| U Utilization | 얼마나 쓰고 있는가 | CPU 사용률, 메모리 사용률 |
| S Saturation  | 얼마나 밀려 있는가 | 실행 대기 큐, 스왑 입출력  |
| E Errors      | 오류가 났는가    | 디스크 오류, 패킷 드롭    |

- **Saturation 이 핵심:** 사용률이 100%가 아니어도 대기가 쌓이면 이미 한계
- **백킹 서비스에 맞음:** 자원을 소비하는 쪽이라 자원 중심으로 봄

사용률만 보면 늦습니다.

CPU 사용률 70%인데 실행 대기가 쌓이고 있으면 이미 포화입니다. 셋을 함께 봐야 「아직 여유가 있다」와 「곧 무너진다」가 갈립니다.



# RED 방법론

| 항목         | 뜻            | 예                |
|------------|--------------|------------------|
| R Rate     | 초당 요청 수      | 초당 명령 수, 초당 질의 수 |
| E Errors   | 실패한 요청 수     | 오류 응답 수, 거부된 연결  |
| D Duration | 요청 하나에 걸린 시간 | 응답 시간 분포         |

- **Duration 은 평균이 아니라 분포:** 평균이 멀쩡해도 꼬리가 길면 누군가는 겪고 있음
- **애플리케이션에 맞춤:** 요청을 받아 처리하는 쪽

평균 응답 시간은 장애를 가립니다.

99%가 1ms 이고 1%가 5초여도 평균은 51ms 입니다. 그 1%가 사용자의 불만이 됩니다.

# USE 와 RED 의 적용 대상

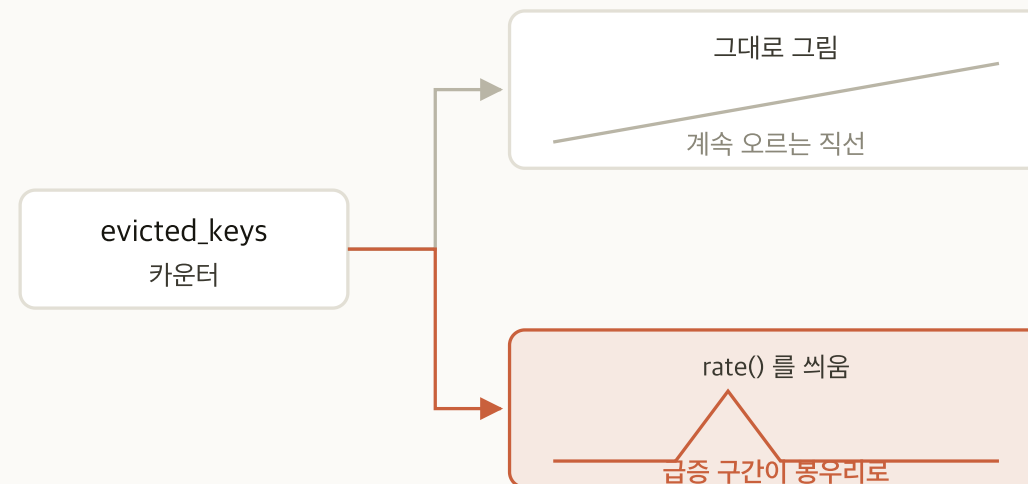
| 대상                      | 쓰는 것      | 이유                   |
|-------------------------|-----------|----------------------|
| 노드 (CPU · 메모리 · 디스크)    | USE       | 자원이 대상               |
| Redis · MongoDB · Kafka | USE 가 주   | 자원을 소비하는 쪽           |
| 세 서비스의 요청 지표            | RED 를 곁들임 | 초당 명령 수, 오류 수, 지연 분포 |
| 애플리케이션                  | RED       | 요청이 대상               |

이 과정은 백킹 서비스가 대상이라 USE 가 중심입니다.

다만 세 서비스도 요청을 받으므로 RED 측의 지표를 함께 씁니다. Redis 의 `instantaneous_ops_per_sec` 와 MongoDB 의 `opLatencies` 가 그 자리입니다.

# 카운터와 게이지

- **게이지**: 지금 값. 오르내림. 메모리 사용량, 연결 수
- **카운터**: 누적 값. **단조 증가만**. 처리한 명령 수, 축출된 키 수
- 카운터를 그대로 그리면: 우상향 직선만 나옴. 아무것도 보이지 않음
- `rate()` 를 씌우면: 단위 시간당 증가량. **이것이 봐야 하는 값**



카운터에 `rate()` 를 씌우지 않으면 장애가 보이지 않습니다.

`evicted_keys` 가 계속 오르는 것은 당연합니다. 봐야 하는 것은 언제부터 빨리 오르기 시작했는가이고, 그것은 증가율에서만 보입니다.

# 03

## 노드 지표와 로그

CPU 와 부하

메모리와 스왑

디스크

공통 장애 넷

로그 경로

# 노드 지표를 먼저 보는 순서

- 서비스 지표에 보이지 않는 것: 디스크 공간 고갈, OOM Killer 의 강제 종료, 노드 전체 정지
- 감별의 첫 갈래: 프로세스만 죽었는가, 노드가 죽었는가
- 세 서비스에 공통: 한 번 익히면 셋에 그대로 적용

서비스 지표가 사라졌다는 것만으로는 원인을 모릅니다.

프로세스가 죽었을 수도, 노드가 죽었을 수도, 네트워크만 끊겼을 수도 있습니다. 세 경우의 대응이 전부 다르므로 노드 지표로 먼저 갈아야 합니다.

# CPU 와 부하

| 지표                               | 확인 위치                                              | 의미            | 판단                           |
|----------------------------------|----------------------------------------------------|---------------|------------------------------|
| <code>node_load1</code> ÷ vCPU 수 | <code>node_exporter</code>                         | 실행 대기 포함 부하   | 1을 넘는 상태가 지속되면 CPU 포화        |
| CPU <code>iowait</code> 비율       | <code>node_cpu_seconds_total{mode="iowait"}</code> | IO 를 기다린 시간   | 높으면 병목이 CPU 가 아니라 <b>디스크</b> |
| CPU <code>steal</code> 비율        | <code>node_cpu_seconds_total{mode="steal"}</code>  | 하이퍼바이저에 뺏긴 시간 | 클라우드에서만 보임. 온프레미스에서<br>는 0   |

`iowait` 이 높으면 CPU 를 늘려도 소용없습니다.

기다리고 있는 것이지 일하고 있는 것이 아닙니다. 이때는 디스크 지표로 넘어갑니다.

# 메모리와 스왑

| 지표                                          | 확인 위치                      | 의미             | 판단                          |
|---------------------------------------------|----------------------------|----------------|-----------------------------|
| <code>node_memory_MemAvailable_bytes</code> | <code>node_exporter</code> | 실제로 쓸 수 있는 메모리 | <code>free</code> 가 아니라 이 값 |
| 스왑 입출력                                      | <code>node_exporter</code> | 스왑으로 나가고 들어온 양 | 0이 아니면 이미 낮음                |

- `free` 를 보면 안 되는 이유: page cache 가 「쓰고 있는 것」으로 잡힘. 필요하면 즉시 반납됨
- `available` 이 답하는 것: 지금 새 프로세스에 줄 수 있는 양
- Kafka 에서 특히: page cache 가 주역이라 `free` 는 거의 항상 작게 보임

Kafka 노드에서 `free` 가 작은 것은 정상입니다.

page cache 가 메모리를 채우고 있는 것이고, 그것이 Kafka 가 빠른 이유입니다. `available` 이 줄어들 때 걱정하면 됩니다.

# 디스크

| 지표                          | 확인 위치              | 의미             | 판단                       |
|-----------------------------|--------------------|----------------|--------------------------|
| node_filesystem_avail_bytes | node_exporter      | 파일 시스템 여유      | Kafka 는 이것이 1순위          |
| 디스크 사용률                     | node_exporter      | 디스크가 일한 시간의 비율 | 1에 가까우면 쉬지 않는 것          |
| 읽기·쓰기 처리량과 IOPS             | node_exporter · 포털 | 실제 처리량         | 디스크 상한과 VM 상한 중 낮은 쪽과 대조 |

- **상한이 둘**: 디스크에 설정한 3,000 IOPS · 125 MB/s 와 VM 의 지속 처리량 상한
- **낮은 쪽이 실제 상한**: E2bds\_v5 는 170 MB/s, D2ds\_v5 는 **85 MB/s**

디스크가 차는 것은 예측할 수 있는 몇 안 되는 장애입니다.  
 갑자기 차지 않고 서서히 줄어듭니다. 남은 용량이 아니라 **줄어드는 속도로** 걸면 대응할 시간이 생깁니다.



# 네트워크 · 파일 디스크립터 · 시계

| 지표          | 확인 위치                                 | 의미           | 판단                           |
|-------------|---------------------------------------|--------------|------------------------------|
| 수신·송신 바이트   | <code>node_exporter</code> · 포털       | 네트워크 처리량     | 복제 지연 감별에 씀                  |
| 열린 파일 디스크립터 | <code>node_exporter</code>            | 프로세스가 연 파일 수 | Kafka 는 segment 마다 파일을 여는 구조 |
| 시계 오차       | <code>node_exporter</code>            | NTP 기준 오차    | 선출 판정과 로그 대조가 여기 걸림          |
| 프로세스 재시작 이력 | <code>journalctl -u &lt;유닛&gt;</code> | 죽고 살아난 흔적    | 지표에는 남지 않음                   |

재시작 이력은 지표에 남지 않습니다.

프로세스가 죽었다 즉시 살아나면 지표 그래프에 잠깐 끊긴 자국만 남고, 왜 죽었는지는 `journalctl` 에만 있습니다. 지표에서 이상한 끊김을 보면 로그로 넘어갑니다.

# 세 서비스 공통 장애 넷

| 증상                      | 판별 근거                                         | 대응                 |
|-------------------------|-----------------------------------------------|--------------------|
| 쓰기가 실패하거나 프로세스가 멈춤      | 파일 시스템 여유. 서비스 지표에는 보이지 않음                    | 오래된 데이터 정리, 디스크 확장 |
| 서비스가 「예고 없이 사라짐」        | 커널 로그의 OOM 기록, <code>journalctl</code> 재시작 이력 | 메모리 상한 재산정, 노드 증설  |
| 노드는 살아 있는데 서비스 간 연결만 끊김 | 노드 지표는 정상, 서비스 지표만 이상                         | 경로 복구, 보안 그룹 확인    |
| 선출이 반복되거나 로그 시각이 맞지 않음  | 시계 오차 지표                                      | 시간 동기화 복구          |

프로세스 정지와 노드 정지는 증상이 다릅니다.

프로세스만 죽으면 `node_exporter` 는 살아 있고, 노드가 죽으면 둘 다 사라집니다. 이 구분이 감별의 첫 갈래입니다.

# 디스크 공간 고갈

```
ops-1: ansible redis -m shell -a "df -h /var/lib/redis"
> ansible redis -m shell -a "df -h /var/lib/redis"
redis-1 /dev/sdc 64G 1.5G 63G 3% /var/lib/redis
redis-2 /dev/sdc 64G 64G 8.0M 100% /var/lib/redis <-- 여유가 바닥
redis-3 /dev/sdc 64G 1.5G 63G 3% /var/lib/redis

> Prometheus
node_filesystem_avail_bytes{mountpoint="/var/lib/redis",
instance="10.0.1.6:9100"} 8388608

# 서비스 지표에는 아무것도 안 나온다. 노드 지표에만 보인다.
# Redis 는 계속 응답하지만 저장에 실패하기 시작한다. 다음 장에서 본다.
```

여유가 바닥에 붙은 그래프와 `df -h` 출력

- **증상:** 쓰기가 오류로 떨어지거나 프로세스가 멈춤
- **무엇으로 아는가:** `node_filesystem_avail_bytes` 가 0에 근접. `ansible all -m shell -a "df -h"` . 각 서비스 로그의 쓰기 실패 원문
- **서비스별 증상:** Redis 는 `MISCONF` 로 쓰기 거부. MongoDB 는 `mongod` 정지. Kafka 는 log dir 이 offline
- **대응:** 오래된 데이터 정리 후 재기동. 근본은 용량 재산정

Kafka 에서 디스크가 차면 Broker 가 멈춥니다.

로그 디렉터리를 못 쓰게 되면 그 Broker 는 요청을 받지 못합니다. Kafka 는 디스크에 쌓는 시스템이라 용량이 다른 두 서비스보다 큼니다.

# OOM Killer

```
redis-3: dmesg -T | grep -i "out of memory"

> ansible redis -b -m shell -a "dmesg -T | grep -i 'out of memory'"
[Sat Aug 29 19:30:40 2026] Out of memory: Killed process 9595 (python3)
total-vms:15686772kB anon-rss:15658112kB file-rss:2240kB
UID:0 pgtables:30716kB oom_score_adj:0

> ansible redis -b -m shell -a "journalctl -u redis-server --since '-10 min'"
(재시작 기록 없음, 이번에는 Redis 가 아니라 별도 프로세스가 죽었다)

# 커널은 메모리를 가장 많이 쓰는 프로세스를 고른다. 실제 운영에서는 그 노드의
# 가장 큰 프로세스가 백킹 서비스이므로 거의 항상 서비스가 죽는다.
# 지표 그래프에는 그 시각에 걸린 자락만 남고, 왜 죽었는지는 여기에만 있다.
```

dmesg 의 OOM 기록과 재시작 이력

- **증상:** 서비스가 예고 없이 사라짐. 지표가 그 시각에 끊김
- **무엇으로 아는가:** dmesg -T 에서 oom 검색. journalctl 의 재시작 이력. 직전에 MemAvailable 이 바닥
- **왜 서비스가 죽는가:** OOM Killer 가 메모리를 가장 많이 쓰는 프로세스를 고름. 백킹 서비스가 그 노드에서 가장 큼
- **대응:** 서비스의 메모리 상한을 노드 메모리보다 충분히 낮게 재산정

Redis 의 maxmemory 를 물리 메모리 가까이 잡으면 이 상황이 옵니다.

스냅샷을 저장하는 순간 fork 로 메모리가 늘어나고, 그 순간 커널이 Redis 를 죽입니다. 챕터 07과 챕터 12에서 그 여유가 왜 필요한지 다시 다룹니다.

# 서비스별 로그 경로와 검색어

| 대상       | 경로                              | 검색 대상                                          |
|----------|---------------------------------|------------------------------------------------|
| Redis    | /var/log/redis/redis-server.log | Background saving, MASTER <=> REPLICAsync, OOM |
| Sentinel | /var/log/redis/sentinel.log     | +sdown, +odown, +switch-master                 |
| MongoDB  | /var/log/mongodb/mongod.log     | "c":"REPL", "c":"ELECTION", Slow query         |
| Kafka    | /opt/kafka/logs/server.log      | ISR, Shrinking, NotEnoughReplicas              |
| systemd  | journalctl -u <유닛> --since      | 기동 실패, 재시작 반복                                  |
| 커널       | dmesg -T                        | oom 으로 검색. 무엇이 죽었는가                            |

로그를 어디서 보는지가 런북 3절의 절반입니다.

새벽에 경로를 찾아 헤매지 않도록 RB-OBS 와 각 런북에 경로를 그대로 적습니다.

# 노드 지표와 로그 점검

- 1 Grafana 노드 대시보드에서 4대의 CPU · 메모리 · 디스크 여유 확인
- 2 `node_load1` 이 가장 높은 노드를 찾고 vCPU 수로 나눠 판단
- 3 ops 노드에서 `ansible all -m shell -a "df -h"` 로 디스크 여유 확인
- 4 Redis 와 Sentinel 로그를 마지막 50줄씩 읽음
- 5 `dmesg -T` 에 OOM 기록이 없는 것 확인

3번 명령의 예상 출력

```
redis-1 | CHANGED | rc=0 >>
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdc         64G   1.3G   63G   2% /var/lib/redis
```

**지금 값이 평상시 값입니다.**

여기서 본 숫자를 적어 둡니다. 챕터 05 에서 `RB-0BS` 의 「정상 범위」 칸에 그대로 들어갑니다.

# 04

## 알람 설계

SLI 와 SLO

증상 기반

세 등급

for 절

억제

recording rule

# 알람의 요건

- **좋은 알람**: 받으면 **할 일이 있음**. 지금 조치하지 않으면 나빠짐
- **나쁜 알람**: 받아도 할 일이 없음. 보다가 익숙해져서 보지 않게 됨
- **판단 기준**: 이 알람을 새벽 3시에 받아도 되는가

**무시되는 알람은 없는 알람입니다.**

울려도 아무도 보지 않는 알람은 걸어 두지 않은 것보다 위험합니다. 진짜 장애가 그 틈에 섞여 같이 무시되기 때문입니다. **알람 수를 줄이는 것도 개선입니다.**



# SLI 와 SLO

| 항목                          | 뜻                | 예             |
|-----------------------------|------------------|---------------|
| SLI Service Level Indicator | 무엇을 측정하는가        | 쓰기 성공률, 응답 시간 |
| SLO Service Level Objective | 어디까지 허용하는가       | 쓰기 성공률 99.9%  |
| 오차 예산                       | SLO 가 허용하는 실패의 양 | 한 달에 43분      |

- **임계값의 근거:** 「메모리 80%」는 어디서 나왔는가에 답하는 것이 SLO
- **없으면:** 임계값이 감으로 정해지고, 알람이 늘어날 근거도 줄어들 근거도 없음

**임계값을 먼저 정하고 근거를 나중에 붙이지 않습니다.**

무엇을 지키려는지 정하면 임계값이 따라 나옵니다. RB-0BS 의 「정상 범위」 칸이 그 결과를 적는 자리입니다.

# 증상 기반과 원인 기반

|          | 증상 기반         | 원인 기반           |
|----------|---------------|-----------------|
| 알람 대상    | 사용자가 겪는 것     | 원인이 될 수 있는 각 지표 |
| 예        | 「쓰기가 거부되고 있다」 | 「메모리 사용률이 높다」   |
| 알람 수     | 적음            | 원인 수만큼 늘어남      |
| 사각지대     | 예상 못 한 원인도 잡힘 | 목록에 없는 원인은 못 잡음 |
| 원인 파악 경로 | 대시보드에서 줍힘     | 알람 자체가 원인       |

증상 하나에 원인이 여섯이면 알람도 여섯이 됩니다.  
 Consumer Lag 은 원인이 여섯 갈래인데 각각에 알람을 걸면 한 장애에 알람이 여섯 번 울립니다. **Lag 이 늘고 있다는 것 하나만 걸고, 여섯 갈래는 챕터 11의 감별로 갑니다.**

# 알람의 세 등급

| 등급   | 뜻         | 적용 시점           | 예                                  |
|------|-----------|-----------------|------------------------------------|
| 페이지  | 사람을 깨움    | 지금 조치하지 않으면 나빠짐 | leader 없는 Partition 발생, PRIMARY 부재 |
| 티켓   | 다음 근무일에 봄 | 지금은 버티지만 곧 문제   | 디스크 여유가 줄어드는 중                     |
| 대시보드 | 알람 없이 봄   | 참고 값            | 초당 명령 수, 처리량                       |

- **셋을 나누지 않으면:** 전부 페이지가 되고, 전부 무시됨
- **나누는 기준:** 지금 사람이 개입하지 않으면 나빠지는가

전부 페이지로 거는 것이 안전한 것이 아닙니다.

깨우는 알람이 많아지면 진짜 깨워야 할 때 반응이 늦어집니다. **페이지로 걸 것을 고를 때는 그 수를 세어 봅니다.**

# for 절

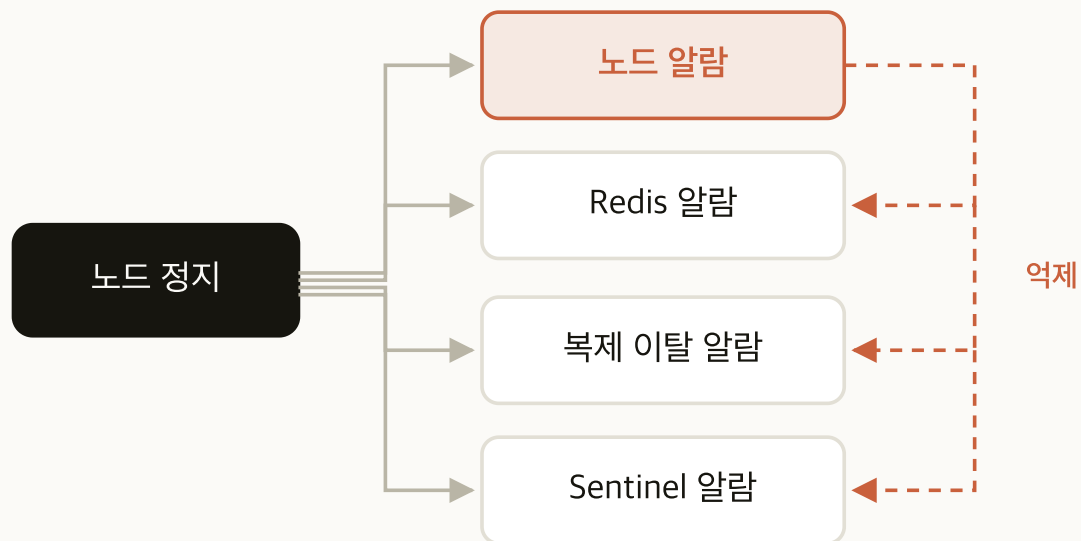
- **하는 일**: 조건이 그 시간만큼 **계속** 참이어야 발동
- **없으면**: 순간적인 급등마다 알람. `scrape` 한 번 실패에도 울림
- **길게 잡으면**: 늦게 알게 됨. **짧게 잡으면**: 사람을 자주 깨우게 됨
- **Pending** 과 **Firing**: 조건은 맞지만 시간이 안 찬 상태가 `Pending`



**for** 값 자체가 운영 판단입니다.

디스크 여유처럼 서서히 나빠지는 것은 길게, leader 상실처럼 즉시 영향이 나는 것은 짧게 잡습니다. 증상이 얼마나 빨리 나빠지는지가 그 값을 정합니다.

# 억제



- **문제:** 노드 한 대가 죽으면 그 위의 알람이 전부 함께 울림
- **억제:** 상위 알람이 발동 중이면 하위 알람을 보내지 않음
- **효과:** 한 장애에 알람 하나. 무엇이 원인인지가 알람 목록에서 바로 보임

**알람 폭발을 줄이는 실제 수단이 억제입니다.**

알람을 지우지 않고도 받는 개수를 줄입니다. 원인 알람은 남아 있고 결과 알람만 묶입니다.

# 페이지로 가는 지표의 기준

| 지표                                       | 등급   | 근거                               |
|------------------------------------------|------|----------------------------------|
| 디스크가 가득 참                                | 페이지  | 쓰기가 그 자리에서 실패                    |
| 서비스 프로세스 정지 ( <code>redis_up</code> 이 0) | 페이지  | 그 노드가 요청을 받지 못함                  |
| OOM Killer 발생                            | 페이지  | 다시 죽을 가능성이 큼                     |
| 디스크 여유가 줄어드는 중                           | 티켓   | <b>아직 시간이 있음.</b> 지금이 아니라 다음 근무일 |
| 스왑 발생                                    | 티켓   | 느려지지만 멈추지는 않음                    |
| 메모리 단편화 상승                               | 티켓   | 추세로 판단할 여유가 있음                   |
| CPU 사용률, 초당 명령 수                         | 대시보드 | 참고 값                             |

# 디스크 알람의 기준: 잔량이 아닌 감소 속도

```
# 남은 양으로 거는 규칙
node_filesystem_avail_bytes{mountpoint="/var/lib/redis"} < 10e9
# 줄어드는 속도로 거는 규칙. 배포된 것의 expr
predict_linear(node_filesystem_avail_bytes{
    mountpoint=~"/var/lib/.*"}[6h], 24 * 3600) < 0
# for: 30m · severity: ticket · runbook: RB-0BS
```

- **남은 양:** 이미 찼을 때만 알려 줌. 대응할 시간이 없음
- **줄어드는 속도:** 지금 추세면 24시간 안에 찬다는 것을 알려 줌

디스크 고갈은 미리 알 수 있는 몇 안 되는 장애입니다.

갑자기 차지 않으므로 추세를 보면 며칠 전에도 알 수 있습니다. 이 규칙은 플레이북이 이미 배포해 두었습니다.

# recording rule

```
groups:
  - name: precompute
    interval: 30s
    rules:
      - record: node:memory_usage:ratio
        expr: 1 - node_memory_MemAvailable_bytes
              / node_memory_MemTotal_bytes
```

- 하는 일: 계산 비용이 큰 질의를 미리 계산해 새 지표로 저장
- 쓰는 곳: 대시보드 여러 곳에서 같은 계산을 반복할 때
- 이름 규칙: 수준:지표:연산 형태로 씀

대시보드가 느려지면 먼저 볼 곳입니다.

패널마다 같은 계산을 매번 하고 있으면 recording rule 로 한 번만 계산합니다.



# Alertmanager

| 기능       | 역할                    | 필요한 이유                  |
|----------|-----------------------|-------------------------|
| Routing  | severity 로 보낼 곳을 나눔   | 페이지는 전화, 티켓은 이슈         |
| Grouping | 같은 원인의 알람을 묶어 한 번에 보냄 | 노드 하나에 알람 열 개가 따로 오지 않게 |
| 억제       | 상위 알람이 있으면 하위를 보내지 않음 | 앞의 그 장치                 |
| 침묵       | 정해진 기간 동안 특정 알람을 멈춤   | 계획된 작업 중에               |

Prometheus 는 알람을 판정하고 Alertmanager 가 전달합니다.

둘의 역할이 나뉘어 있어 규칙을 고치지 않고도 보내는 방식을 바꿀 수 있습니다.

# 05

## 장애 대응 절차와 런북

대응 다섯 단계

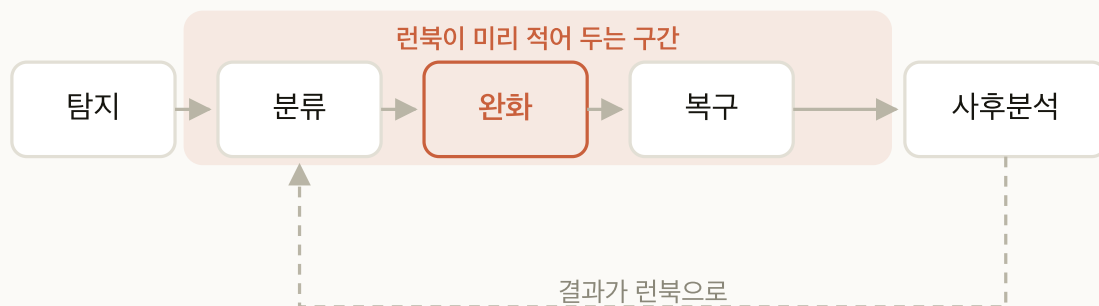
완화 우선

런북 8절

새벽 3시 테스트

RB-OBS

# 장애 대응 다섯 단계



| 단계   | 내용                |
|------|-------------------|
| 탐지   | 알람 또는 신고          |
| 분류   | 영향 범위와 심각도 판정     |
| 완화   | 서비스를 먼저 살림        |
| 복구   | 정상 상태로 되돌림        |
| 사후분석 | 왜 일어났고 무엇을 바꿀 것인가 |

**런북은 분류·완화·복구를 미리 적어 둔 것입니다.**

장애가 났을 때 처음부터 생각하지 않도록, 겪기 전에 적어 두는 것이 런북입니다.

# 완화와 복구의 순서

|          | 완화                | 복구                    |
|----------|-------------------|-----------------------|
| 목적       | 서비스를 먼저 살림        | 정상 상태로 되돌림            |
| 원인 파악 여부 | 몰라도 됨             | 알아야 함                 |
| 예        | 재시작, 읽기 차단, 유량 제어 | 설정 재산정, 인덱스 추가, 용량 증설 |
| 걸리는 시간   | 짧음                | 김                     |

원인을 다 알기 전에도 할 수 있는 일이 있습니다.

replica 하나가 빠진 것을 보면 원인을 몰라도 그 노드를 재기동해 볼 수 있습니다.

완화가 증거를 지울 수 있습니다.

재시작하면 메모리 상태와 스택이 사라집니다. 되돌리기 전에 로그와 지표를 남깁니다.

# 런북 8절 구조

| 절 | 내용                  |
|---|---------------------|
| 1 | 증상                  |
| 2 | 영향 범위               |
| 3 | 즉시 확인. 3분 안에 돌릴 명령만 |
| 4 | 원인 감별               |
| 5 | 복구 절차               |
| 6 | 복구 검증               |
| 7 | 예방                  |
| 8 | 관련 지표               |

**3절에 설명을 섞지 않습니다.**

「즉시 확인」은 잠결에 복사해서 붙이는 자리입니다. 왜 그 명령인지는 4절에서 설명합니다. 섞이면 급할 때 읽는 시간이 늘어납니다.

# 새벽 3시 테스트

- 복사해서 바로 붙일 수 있는가: 노드 이름과 경로가 채워져 있는가
- 각 단계에 성공 판정이 있는가: 다음으로 넘어가도 되는지 알 수 있는가
- 되돌릴 수 없는 명령에 경고가 있는가: 대안이 함께 적혀 있는가
- 전제가 적혀 있는가: 어느 노드에서, 어느 디렉터리에서

「상황에 따라 판단합니다」는 런북에 쓰지 않습니다.

판단이 필요한 자리라면 판단 기준을 숫자로 적습니다. 「지연이 oplog 윈도우의 절반을 넘으면 5절로, 아니면 6절로」처럼 갈림길을 명시합니다.

# RB-OBS 와 장애 대응 런북의 구조 차이

|        | RB-OBS                                     | 장애 대응 런북 넷 |
|--------|--------------------------------------------|------------|
| 진입점    | 평상시 점검                                     | 겪고 있는 증상   |
| 답하는 질문 | 지금 정상인가                                    | 어떻게 되돌리는가  |
| 구조     | 지표 · 확인 위치 · 정상 범위 · 확인 주기 · <b>넘어갈 런북</b> | 위의 8절      |
| 사용 시점  | 당번이 아침에, 교대할 때                             | 알람을 받았을 때  |

마지막 칸이 둘을 잇습니다.

RB-OBS 에서 지표가 정상 범위를 벗어난 것을 보면 그 줄이 어느 런북으로 갈지 알려 줍니다. 런북 이름이 적혀 있어야 지표 확인이 대응으로 이어집니다.

## RB-OBS 배대 작성

- 1 ops 노드에 `~/runbook` 을 만들고 그 안에 런북 파일을 만들
- 2 표에 다섯 열을 만들: 지표 / 확인 위치 / 정상 범위 / 확인 주기 / 넘어갈 런북
- 3 **노드 공통 지표부터 채움.** 챕터 03 에서 적어 둔 평상시 값이 정상 범위가 됨
- 4 확인 위치에 대시보드 이름과 패널 이름, 또는 명령을 그대로 적음
- 5 「넘어갈 런북」 칸은 **비워 둬.** 챕터 07 · 09 · 11 에서 서비스마다 채움

**확인 위치를 「Grafana」로만 적지 않습니다.**

어느 대시보드의 어느 패널인지까지 적어야 새벽에 찾지 않습니다.



# RB-OBS 노드 지표 칸

| 지표                             | 확인 위치                                        | 정상 범위   | 주기     | 넘어갈 런북      |
|--------------------------------|----------------------------------------------|---------|--------|-------------|
| 디스크 여유                         | 노드 대시보드 · 디스크 패널                             | 실측으로 채움 | 교대마다   | 챕터 07 부터 채움 |
| <code>MemAvailable</code>      | 노드 대시보드 · 메모리 패널                             | 실측으로 채움 | 교대마다   |             |
| <code>node_load1</code> ÷ vCPU | 노드 대시보드 · CPU 패널                             | 실측으로 채움 | 교대마다   |             |
| 스왑 입출력                         | 노드 대시보드 · 메모리 패널                             | 0       | 교대마다   |             |
| OOM 기록                         | <code>dmesg -T</code> 에서 <code>oom</code> 검색 | 없음      | 하루 한 번 |             |

「정상 범위」에 숫자가 없으면 점검이 되지 않습니다.

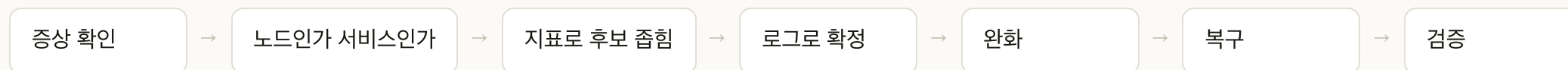
「낮게 유지」는 판단을 요구합니다. 챕터 03 에서 본 값을 근거로 숫자를 넣습니다.

# 운영 절차서와 런북

|       | 운영 절차서                | 런북              |
|-------|-----------------------|-----------------|
| 진입점   | 하려는 작업                | 겪고 있는 증상        |
| 예     | 「Sentinel 을 구성한다」     | 「replica 가 빠졌다」 |
| 판단 여지 | 있음. 값을 정하며 진행         | 없음. 그대로 따라 하면 됨 |
| 출처    | 세션 1 부터 세션 3 에서 남긴 기록 | 이 세션에서 새로 씀     |

절차서의 명령이 런북 5절의 재료가 됩니다.  
 그대로 옮기는 것이 아니라 **판단 여지를 없앤 형태로 다시 씁니다**. 절차서의 「적절한 값을 정합니다」는 런북에서 「이 값을 넣습니다」가 되어야 합니다.

# 장애를 만났을 때 보는 순서



- 첫 갈래는 계층: 노드 지표가 정상인가 아닌가
- 둘째 갈래는 서비스별: 그 서비스의 대표 장애 목록에서 후보를 고름
- 확정은 로그: 지표가 좁혀 준 뒤 로그가 원인을 확정

이 순서가 챕터 06 부터 챕터 11에서 세 번 반복됩니다.

Redis · MongoDB · Kafka 에서 같은 모양으로 진행하므로, 세 번째에는 다음에 무엇이 올지 알고 따라오게 됩니다.

# 이 세션이 다루는 장애의 범위

| 서비스      | 다루는 장애 | 재현 대상           |
|----------|--------|-----------------|
| 세 서비스 공통 | 4      | 없음. 지표와 대응만     |
| Redis    | 12     | 복제 이탈, 메모리 압박   |
| MongoDB  | 11     | 복제 지연           |
| Kafka    | 11     | Consumer Lag 급증 |
| 합계       | 38     | 런북 4건           |

재현하지 않는 장애도 슬라이드에 전부 있습니다.

이 자료는 강의가 끝난 뒤 참조 런북으로 쓰입니다. 증상을 만났을 때 여기서 찾아 감별과 대응까지 갈 수 있어야 하므로, 다루는 깊이와 실는 범위를 다르게 두었습니다.

# 06

## Redis 지표와 알람 규칙

메모리 지표

복제 지표

지연 지표

PromQL

규칙 작성

# Redis 지표의 출처

- **INFO memory** : 메모리 사용량, 단편화, 정책
- **INFO stats** : 히트율, 축출된 키, fork 시간, 재동기화 횟수
- **INFO replication** : master 연결 상태, replica 수, 복제 offset
- **INFO clients** : 연결 수, 블로킹 대기
- **SLOWLOG GET** : 느린 명령의 원문. **지표가 아니라 목록**

**SLOWLOG** 는 대시보드에 나오지 않습니다.

느린 명령이 무엇이었는지는 지표에 나오지 않고 그 자리에서 조회해야 합니다. 느린 명령 하나가 전체를 막는 Redis 1순위 장애의 범인을 찾는 명령이므로 런북 3절에 넣습니다.

# 메모리 지표

| INFO 필드                                           | Prometheus 지표                                                              | 의미                             | 판단                                     |
|---------------------------------------------------|----------------------------------------------------------------------------|--------------------------------|----------------------------------------|
| <code>used_memory</code> ÷ <code>maxmemory</code> | <code>redis_memory_used_bytes</code> ÷ <code>redis_memory_max_bytes</code> | 메모리 사용률                        | 80% 이상이면 eviction 임박                   |
| <code>maxmemory_policy</code>                     | <code>redis_instance_info</code> 의 label                                   | 한계 도달 시 동작                     | <code>noeviction</code> 이면 쓰기가 거부됨     |
| <code>evicted_keys</code> 증가율                     | <code>redis_evicted_keys_total</code>                                      | 축출된 키 수                        | 0이 아니면 캐시가 모자람                         |
| <code>expired_keys</code> 증가율                     | <code>redis_expired_keys_total</code>                                      | TTL 로 사라진 키 수                  | <code>evicted_keys</code> 와 구분. 이쪽은 정상 |
| <code>mem_fragmentation_ratio</code>              | <code>redis_mem_fragmentation_ratio</code>                                 | RSS ÷ <code>used_memory</code> | 1.5 이상은 단편화, 1 미만은 스왑                  |

`evicted_keys` 와 `expired_keys` 를 섞으면 안 됩니다.

TTL 로 사라지는 것은 설계대로 도는 것이고, 축출되는 것은 메모리가 모자란 것입니다. 둘을 합쳐 보면 정상 동작을 장애로 읽습니다.

# Persistence 와 fork 지표

| INFO 필드                                  | Prometheus 지표                                  | 의미               | 판단                         |
|------------------------------------------|------------------------------------------------|------------------|----------------------------|
| <code>latest_fork_usec</code>            | <code>redis_latest_fork_seconds</code>         | 마지막 fork 에 걸린 시간 | 커지면 저장할 때마다 멈춤             |
| <code>rdb_last_bgsave_status</code>      | <code>redis_rdb_last_bgsave_status</code>      | 마지막 스냅샷 성공 여부    | 실패가 쌓이면 백업이 없는 상태          |
| <code>rdb_changes_since_last_save</code> | <code>redis_rdb_changes_since_last_save</code> | 마지막 저장 이후 변경 수   | 유실 가능 구간의 크기               |
| <code>aof_last_write_status</code>       | <code>redis_aof_last_write_status</code>       | AOF 쓰기 성공 여부     | <code>err</code> 이면 디스크를 봄 |

fork 는 Redis 를 잠깐 멈춥니다.

BGSAVE 가 프로세스를 fork 하는 순간 전체가 멈추고, 데이터가 클수록 그 시간이 길니다. `latest_fork_usec` 이 커지는 것은 저장할 때마다 지연이 된다는 뜻입니다.



# 복제 지표

| INFO 필드                     | Prometheus 지표                                         | 의미                    | 판단               |
|-----------------------------|-------------------------------------------------------|-----------------------|------------------|
| connected_slaves            | redis_connected_slaves                                | master 가 본 replica 수  | 구성값보다 적으면 이탈     |
| master_link_status          | redis_master_link_up · redis_up                       | 연결 상태와 Redis 응답       | 0이면 끊겼거나 죽은 것    |
| 복제 offset 의 차               | redis_connected_slave_offset_bytes                    | 각 replica 의 복제 offset | 차가 벌어지면 못 따라잡는 것 |
| role                        | redis_instance_info 의 label                           | master 인가 replica 인가  | 알람을 줄힐 때         |
| sync_full · sync_partial_ok | redis_replica_resyncs_full · _partial_resync_accepted | 전체·부분 재동기화 횟수         | full 이 반복되면 부족   |
| master_sync_in_progress     | redis_master_sync_in_progress                         | 재동기화 중인가              | 1이면 기다림          |

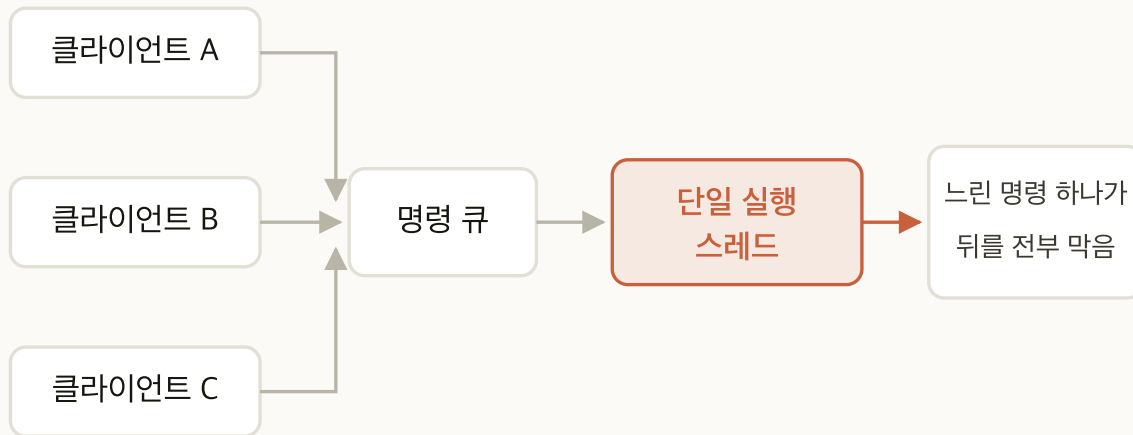
# 연결과 지연 지표

| INFO 필드                                                  | Prometheus 지표                                                               | 의미          | 판단                |
|----------------------------------------------------------|-----------------------------------------------------------------------------|-------------|-------------------|
| blocked_clients                                          | redis_blocked_clients                                                       | 블로킹 명령 대기 수 | 느린 명령의 신호         |
| <code>connected_clients</code> ÷ <code>maxclients</code> | <code>redis_connected_clients</code> ÷ <code>redis_config_maxclients</code> | 연결 사용률      | 상한에 가까우면 거부 임박    |
| rejected_connections                                     | redis_rejected_connections_total                                            | 거부된 연결 수    | 0이 아니면 상한에 닿은 것   |
| keyspace_hits 비율                                         | <code>redis_keyspace_hits_total</code> ÷ <code>_misses_total</code>         | 히트율         | 떨어지면 캐시 효용이 없어진 것 |
| instantaneous_ops_per_sec                                | 없음. <code>redis_commands_processed_total</code>                             | 초당 명령 수     | 급락이 급증보다 위험한 신호   |
| 명령별 호출·지연                                                | <code>redis_commands_total</code> ÷ <code>_duration_seconds_total</code>    | 명령별 지연      | 전체를 막는 느린 명령의 범인  |

초당 명령 수가 갑자기 떨어지면 Redis 가 막힌 것입니다.

느린 명령 하나가 전체를 멈춰 세운 경우도 같은 모양입니다. `blocked_clients` 와 `SLOWLOG` 를 함께 봐야 갑니다.

# Redis 의 단일 실행 스레드



- 명령 처리는 한 번에 하나: 다중 코어를 쓰지 않음
- 그래서 빠름: 잠금이 없고 문맥 전환이 없음
- 그래서 위험함: 오래 걸리는 명령 하나가 모든 클라이언트를 멈춰 세움
- 오래 걸리는 것들: 전체 키 순회, 큰 컬렉션 삭제, 오래 도는 스크립트

Redis 운영 사고에서 가장 자주 나오는 모양입니다.

CPU 도 메모리도 멀쩡한데 모든 클라이언트가 동시에 느려집니다. 자원 지표만 보면 원인이 드러나지 않고, SLOWLOG 를 봐야 나옵니다.

# PromQL 기초

Prometheus 의 **Graph** 화면에서 실행합니다

```
# label 로 고르기
node_memory_MemAvailable_bytes{service="redis"}

# 카운터의 증가율
rate(redis_evicted_keys_total[5m])

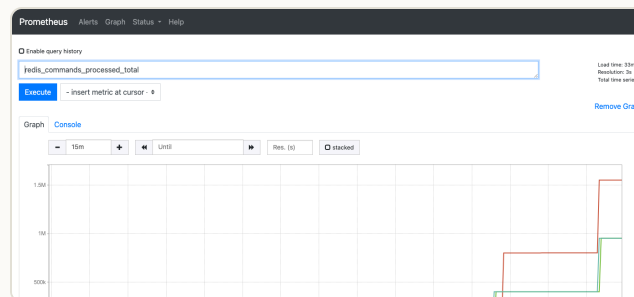
# 노드별 값을 하나로 모으기
avg by (instance) (
    rate(node_cpu_seconds_total{mode="idle"}[5m]))

# 두 지표의 비
redis_memory_used_bytes / redis_memory_max_bytes
```

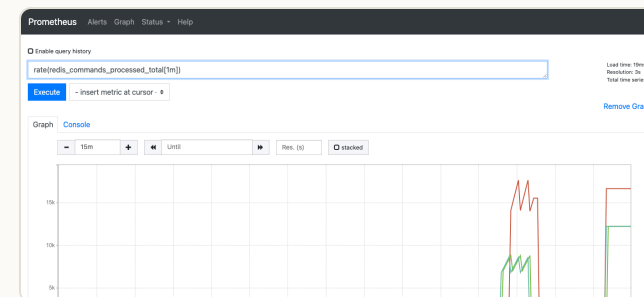
- **{}** : label 로 대상 좁히기. **service** label이 여기서 쓰임
- **[5m]** : 최근 5분 구간. **rate()** 와 함께 씀
- **by (instance)** : 노드별로 나눠 집계

# rate() 유무 비교

- 1 Prometheus 의 Graph 화면에서 카운터 지표를 그대로 조회
- 2 같은 지표에 rate(...[5m]) 을 씌워 조회
- 3 두 그래프의 모양을 비교



그대로 그린 그래프. 우상향 직선



rate() 를 씌운 그래프. 증감이 보임

카운터를 그대로 그린 그래프로는 장애 시점을 못 찾습니다.  
값이 계속 오르는 것은 당연하고, 봐야 하는 것은 언제부터 빨리 오르기 시작했는가입니다.

# 노드와 Redis 지표 질의

- 1 노드 메모리 여유를 조회
- 2 Redis 메모리 사용률을 비율로 조회
- 3 축출된 키 수의 증가율을 `rate()` 로 조회
- 4 각 결과를 Grafana 대시보드의 같은 패널 값과 대조

**지표 이름이 문서와 다를 수 있습니다.**

exporter 마다 이름 규칙이 달라 `INFO` 의 필드 이름이 그대로 지표 이름이 되지 않습니다. 결과가 비면 Prometheus 의 지표 목록에서 실제 이름을 먼저 확인합니다.

# 알람 규칙의 구조

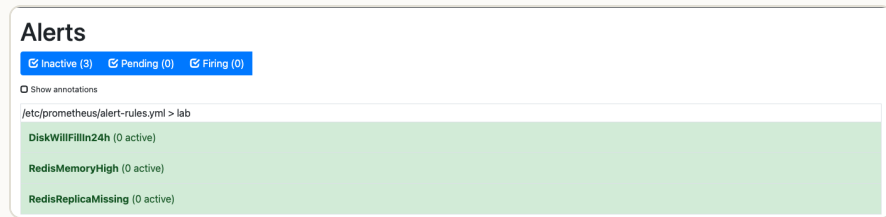
```
- alert: RedisReplicaMissing
  expr: redis_connected_slaves < 2
    and on(instance)
    redis_instance_info{role="master"}
  for: 2m
  labels: { severity: page }
  annotations:
    summary: "replica 가 구성값보다 적습니다"
    runbook: "RB-R01"
```

- **expr** : 참이면 조건 충족. **좁히지 않으면** replica 의 값이 0이라 항상 발동
- **for** : 그 시간만큼 계속 참이어야 발동
- **labels.severity** : 페이지 · 티켓 · 대시보드 중 하나
- **annotations.runbook** : 받은 사람이 펼 런북 이름

알람에 런북 이름을 붙입니다.

새벽에 알람을 받은 사람이 무엇을 볼지 찾는 시간이 대응 시간에 그대로 더해집니다. 알람과 런북이 이어져 있어야 **RB-OBS** 없이도 대응으로 바로 이어집니다.

# 알람 규칙 두 건 작성



Alerts 화면. 세 규칙이 Inactive

- 1 **디스크:** 이미 들어 있는 규칙에서 `mountpoint` 와 `for` 의 이유를 확인
- 2 **Redis 메모리:** 사용률이 임계값을 넘을 때. **직접 작성.** `maxmemory` 값 위에서 잡음
- 3 **Redis 복제:** `connected_slaves` 가 구성값보다 적을 때. **직접 작성**
- 4 각 규칙에 `for` 절과 `severity` label, `runbook` 주석 부여
- 5 규칙 파일을 반영하고 Alerts 화면에서 확인

대상을 좁히지 않으면 규칙이 계속 발동한 채로 남습니다.

replica 의 `redis_connected_slaves` 는 항상 0 입니다. `and on(instance) redis_instance_info{role="master"}` 를 붙입니다.

지금 없는 지표로는 규칙을 검증할 수 없습니다.

나머지 서비스의 규칙은 그 지표를 배운 직후에 씁니다.



## 등급 분류와 페이지 수 점검

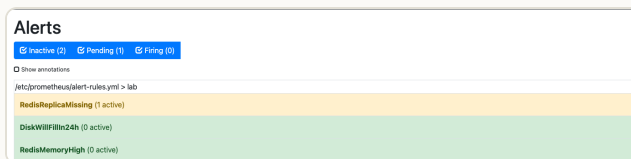
- ☐ 세 규칙에 `severity` 부여
- ☐ 그 수를 정한 근거를 설명할 수 있음
- ☐ 각 규칙에 `runbook` 주석 부여
- ☐ 페이지로 분류한 규칙 수를 셈
- ☐ 티켓으로 내린 규칙이 왜 페이지가 아닌지 설명할 수 있음
- ☐ `for` 값을 규칙마다 다르게 잡은 이유를 설명할 수 있음

세 규칙이 모두 페이지라면 다시 봅니다.

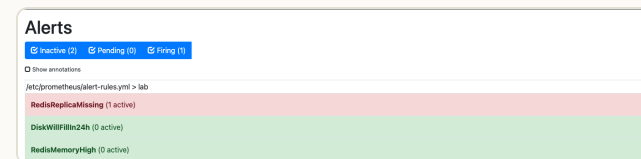
전부 깨울 만한 것인지, 아니면 나누는 기준이 아직 안 선 것인지 판단합니다. 기준은 「지금 사람이 개입하지 않으면 나빠지는가」입니다.

# 알람 상태 전이 확인

- 1 챕터 07 부터 챕터 11의 장애 재현 중에 Alerts 화면을 열어 둠
- 2 조건이 참이 되는 순간 Pending 으로 바뀌는 것 확인
- 3 for 시간이 지나 Firing 으로 넘어가는 것 확인
- 4 조치 후 Inactive 로 돌아오는 것 확인



조건은 참인데 시간이 안 찬 Pending



시간이 지나 Firing 으로 넘어간 상태

이 확인은 뒤 챕터의 장애 재현과 함께 합니다.

알람을 위해 따로 장애를 만들지 않습니다. 뒤 챕터에서 재현할 장애를 관찰하면서 함께 확인합니다.

# 07

## Redis 대표 장애와 재현

대표 장애 12

느린 명령

big key

복제 이탈 재현

메모리 압박 재현

# Redis 대표 장애 (1)

| 증상                               | 판별 근거                                            | 대응                          |
|----------------------------------|--------------------------------------------------|-----------------------------|
| replica 하나가 빠짐                   | master_link_status 가 down , connected_slaves 감소  | 노드 복구 후 자동 재접속 확인           |
| 쓰기가 OOM command not allowed 로 실패 | 메모리 사용률 1.0, 정책이 noeviction                      | 정책 조정 또는 용량 재산정             |
| 모든 클라이언트가 동시에 멈춤                 | SLOWLOG , blocked_clients , 초당 명령 수 급락           | 느린 명령 중단. 순회 명령을 대체 명령으로 교체 |
| 히트율이 급락하고 백엔드 부하가 오름             | evicted_keys 증가율, 히트율                            | maxmemory 재산정, TTL 설계 점검    |
| 특정 키 접근에서만 지연                    | --bigkeys , MEMORY USAGE . --hotkeys 는 LFU 정책에서만 | 큰 값을 나눔. hot key 를 분산       |
| 저장할 때마다 멈추고 스냅샷이 실패              | rdb_last_bgsave_status , latest_fork_usec        | maxmemory 를 fork 여유까지 낮춤    |

# Redis 대표 장애 (2)

| 증상                           | 판별 근거                                                                                               | 대응                                          |
|------------------------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------|
| 재접속마다 master 의 CPU 와 디스크가 급등 | <code>sync_full</code> 증가, <code>repl-backlog-size</code> 대비 지연                                     | <code>repl-backlog-size</code> 상향           |
| 멀쩡한 master 가 강등됨             | <code>sentinel.log</code> 에 <code>+odown</code> 과 <code>+switch-master</code> 가 있는데 그 시각의 노드 지표는 정상 | <code>down-after-milliseconds</code> 상향     |
| 지연의 꼬리가 길어짐                  | <code>mem_fragmentation_ratio</code> 가 1 미만, 노드 스왑 입출력                                              | 부하 감소, 노드 메모리 재산정                           |
| TTL 동시 만료 직후 miss 가 급증       | miss 급증에 백엔드 부하가 동반됨                                                                                | TTL 에 <code>jitter</code> 를 줌. 재계산을 한 곳에 서만 |
| 새 연결이 거부됨                    | <code>connected_clients</code> ÷ <code>maxclients</code> , <code>rejected_connections</code>        | 커넥션 풀 설정 점검, <code>maxclients</code> 상향     |
| 쓰기가 오류로 떨어지고 AOF 가 밀림        | <code>aof_last_write_status</code> , 파일 시스템 여유                                                      | 디스크 정리 후 재기동                                |

# 느린 명령 전면 블로킹

```
ops-1: redis-cli -h 10.0.1.5 SLOWLOG GET 3

> CONFIG SET slowlog-log-slower-than 10000 # 10ms 를 넘는 명령을 기록
> SLOWLOG RESET
> SLOWLOG GET 3

1) 1) (integer) 7 # 일련번호
2) (integer) 1788030135 # 실행 시각
3) (integer) 26068 # 걸린 시간 (마이크로초) = 26ms
4) 1) "LRANGE"
2) "big:1ist"
3) "0"
4) "-1"
5) "10.0.1.4:56240"

2) 1) (integer) 6
3) (integer) 11263 # 11ms
4) 1) "KEYS"
2) "*"
5) "10.0.1.4:56224"

# 항목 20만 개짜리 list 를 통째로 읽는 데 26ms 가 걸렸다. 그동안 다른 명령은
# 전부 대기한다. 같은 시각 instantaneous_ops_per_sec 이 9 로 떨어져 있다.
```

SLOWLOG GET 과 초당 명령 수 급락

- **증상:** CPU 도 메모리도 정상인데 모든 클라이언트가 동시에 느려짐
- **무엇으로 아는가:** SLOWLOG GET 10 에 오래 걸린 명령이 남음.  
blocked\_clients 가 오름. instantaneous\_ops\_per\_sec 이 그 시각에 떨어짐
- **원인이 되는 명령:** 전체 키 순회, 큰 컬렉션 삭제, 오래 도는 스크립트
- **대응:** 순회 명령을 커서 기반 명령으로 교체. 큰 값은 나눠서 지움

운영 중인 Redis 에서 전체 키를 순회하는 명령을 쓰지 않습니다.

키가 많을수록 오래 걸리고, 그동안 모든 클라이언트가 멈춥니다. 커서 기반 명령은 나눠 돌므로 중간에 다른 명령이 끼어들 수 있습니다.

# big key 와 hot key

```
ops-1: redis-cli -h 10.8.1.5 --bigkeys

----- summary -----
Sampled 95039 keys in the keyspace!
Total key length in bytes is 1520608 (avg len 16.00)

Biggest list found "big:list" has 200000 items
Biggest hash found "big:hash" has 50000 fields
Biggest string found "key:_rand_int_" has 512 bytes

1 lists with 200000 items (00.00% of keys, avg size 200000.00)
1 hashes with 50000 fields (00.00% of keys, avg size 50000.00)
95037 strings with 24329728 bytes (100.00% of keys, avg size 256.00)

# 9만 5천 키 가운데 항목 20만 개짜리 list 하나와 필드 5만 개짜리 hash 하나가
# 있다. 값을 읽고 쓰는 것보다 지울 때가 위험하다. 큰 컬렉션을 한 번에 지우면
# 그동안 Redis 가 멈춘다.
```

--bigkeys 출력. 타입별 가장 큰 키

- **big key**: 값 하나가 큰 키. 읽고 쓰고 지우는 데 오래 걸림
- **hot key**: 접근이 몰리는 키. 값은 작아도 그 키만 부하가 큼
- **무엇으로 아는가**: `--bigkeys` , `--hotkeys` , `MEMORY USAGE <키>`
- **대응**: 큰 값을 여러 키로 나눔. hot key 는 읽기를 replica 로 분산

`--hotkeys` 는 `maxmemory-policy` 가 LFU 계열일 때만 동작합니다.  
LRU 정책에서는 접근 빈도를 세지 않아 결과가 나오지 않습니다.

**big key 는 그 자체보다 지울 때가 위험합니다.**

큰 컬렉션을 한 번에 지우면 그동안 Redis 가 멈춥니다. 앞 장의 전면 블로킹이 여기서 시작됩니다.

# fork 실패와 저장 지연

```
ops-1 : 디스크가 한 노드에서 BGSAVE
> redis-cli -h 10.0.1.6 BGSAVE
Background saving started

> redis-cli -h 10.0.1.6 INFO persistence
rdb_bgsave_in_progress 0
rdb_last_bgsave_status err <-- 실패

> tail /var/log/redis/redis-server.log
* Background saving started by pid 9714
# Write error while saving DB to the disk(rdbSaveRio):
# No space left on device
# Background saving error

# 지표는 err 한 글자만 준다. 왜 실패했는지는 로그의 원문에만 있다.
# 'No space left on device'가 알 장의 디스크 여유와 이어진다.
```

rdb\_last\_bgsave\_status:err 와 로그 원문

- **증상:** 스냅샷이 남지 않음. 저장 시각마다 지연이 급등
- **무엇으로 아는가:** `rdb_last_bgsave_status` 가 `err`.  
`latest_fork_usec` 이 큼. 로그에 저장 실패 원문. 노드의 `MemAvailable` 이 그 시각에 바닥
- **왜 그런가:** `BGSAVE` 가 fork 하고, 부모가 쓰는 페이지가 복사되어 메모리가 최대 두 배까지 늘 수 있음
- **대응:** `maxmemory` 를 물리 메모리의 절반 근처로 낮춤.  
`vm.overcommit_memory` 확인

`maxmemory` 를 물리 메모리 가까이 잡으면 저장할 때 죽습니다.

남겨 둔 절반은 낭비가 아니라 fork 여유입니다. 세션 1 에서 커널 설정을 만진 이유가 이것이고, 챕터 12에서 세 서비스를 비교할 때 다시 나옵니다.



# Sentinel 오탐 전환

```
ops-1: ansible redis -b -a "tail /var/log/redis/sentinel.log"

# down-after-milliseconds 를 300 으로 줄이고 replica 한 대를 12초 멈춘 뒤

> grep -oE "[a-z-]+ (slave|master)" /var/log/redis/sentinel.log | sort | uniq -c
    6 +sdown slave      <-- 주관적 정지 판정. Sentinel 셋이 각각 냈다
    3 +reboot slave     <-- 다시 응답하기 시작
    3 +set master       <-- 판정 시간을 되돌린 기록

+odown 없음
+switch-master 없음

# +sdown 은 Sentinel 하나의 주관적 판정이고 +odown 은 정족수가 모인 판정이다.
# +sdown 만 반복되고 +odown 이 나오지 않으면 판정 시간이 너무 짧은 것이다.
# 여기서는 replica 라 전환이 일어나지 않았다. master 였다면 +odown 뒤에
# +switch-master 가 따라온다.
```

sentinel.log 의 판정과 전환 기록

- **증상:** 멀쩡한 master 가 강등되고 전환이 일어남
- **+sdown** 만 있고 **+odown** 이 없음: Sentinel 하나만 정지로 본 상태
- **+odown** 뒤 **+switch-master**: 정족수가 모여 전환이 실제로 일어남
- **왜 그런가:** **down-after-milliseconds** 가 짧으면 잠깐의 지연을 정지로 판정
- **대응:** 판정 시간을 늘림. 네트워크 지연의 실제 분포를 보고 정함

**+sdown** 과 **+odown** 은 다릅니다.

앞은 Sentinel 하나의 주관적 판정이고, 뒤는 정족수가 모인 객관적 판정입니다. **+sdown** 만 반복되고 **+odown** 이 나오지 않으면 판정 시간이 너무 짧은 것입니다.

# 캐시 스템피드와 연결 고갈

- **캐시 스템피드의 증상:** TTL 이 동시에 만료된 직후 miss 가 급증하고 백엔드 부하가 함께 오름
- **무엇으로 아는가:** 히트율 급락에 **백엔드 부하 상승이 동반됨**
- **대응:** TTL 에 `jitter` 를 줌. 재계산을 한 곳에서만 하도록 잠금
- **연결 고갈:** 새 연결이 거부됨. `rejected_connections` 가 0이 아님. 커넥션 풀 설정 점검과 상한 상향

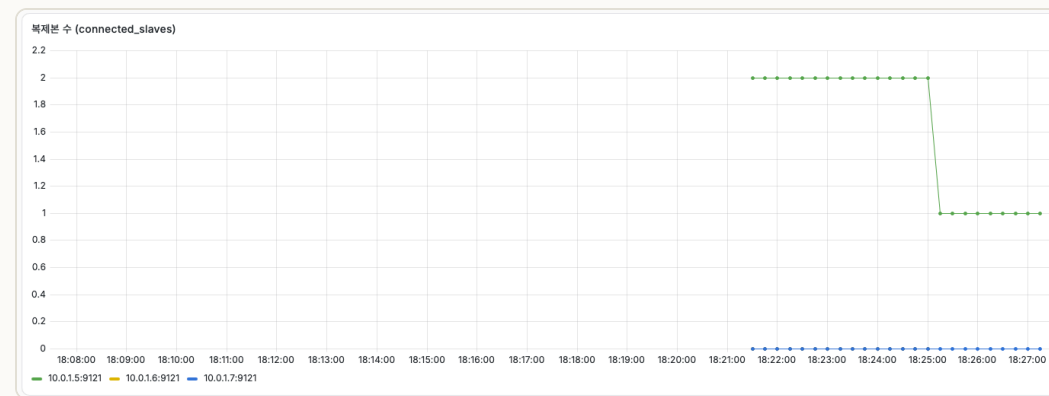


히트율 하락만으로는 원인이 갈리지 않습니다.

`evicted_keys` 가 함께 오르면 캐시가 모자란 것이고, 오르지 않고 miss 만 튀면 TTL 동시 만료입니다. 두 경우 모두 백엔드 부하가 오르므로 그것으로는 갈리지 않습니다.

# 장애: replica 하나 이탈

- 1 ops 노드에서 `cd ~/lab/scenarios`
- 2 `./rb-r01.sh` 실행
- 3 Grafana 의 「replica 수」 패널에서 값이 2에서 1로 떨어지는 것 확인
- 4 Alerts 화면에서 복제 규칙이 Pending 을 거쳐 Firing 이 되는 것 확인
- 5 이 시각과 두 화면을 적어 둬. **런북 1절의 재료**



「replica 수」 패널. 값이 2에서 1로

알람이 울리는 것까지가 증상입니다.

앞서 건 규칙이 여기서 실제로 발동합니다.

~/scenarios 의 rb- 로 시작하는 네 스크립트가 장애를 주입합니다. --restore 로 되돌리고 --status 로 지금 상태를 봅니다. 무엇을 바꾸는지는 열어 보지 않고 지표로 찾습니다. 완료 판정: 패널 값이 1이고 복제 규칙이 Firing 입니다.

# 원인 후보 다섯

| 원인              | 판별 근거                                              | 대응                   |
|-----------------|----------------------------------------------------|----------------------|
| replica 프로세스 정지 | redis_up 이 0. 두 exporter 의 target 은 UP 유지          | 재기동                  |
| 노드 정지           | 두 exporter 의 target 이 함께 DOWN                      | 노드 복구                |
| 네트워크 차단         | 노드는 살아 있으나 복제만 끊김                                  | 경로 복구                |
| 전체 재동기화 진행 중    | master_sync_in_progress 가 1, master 의 CPU 와 디스크 상승 | 기다림                  |
| repl-backlog 부족 | sync_full 이 반복해서 오름                                | repl-backlog-size 상향 |

첫 갈래는 노드인가 서비스인가입니다.

챕터 03에서 배운 그 갈래입니다. redis\_up 이 0인가 target 이 DOWN 인가로 프로세스 정지와 노드 정지가 갈립니다.

# 감별 절차

- 1 ops 노드에서 `redis-cli -h <master 사실 IP> INFO replication`
- 2 `connected_slaves` 와 각 replica 줄에서 빠진 노드를 찾음
- 3 Prometheus 에서 `redis_up` 과 `up{job="redis"} · up{job="node"}` 를 함께 조회
- 4 `redis-cli -h <replica 사실 IP> INFO replication` 으로 `master_link_status` 확인
- 5 `master_sync_in_progress` 로 재동기화 중인지 확인
- 6 `sync_full` 과 `sync_partial_ok` 를 비교해 반복 여부 확인
- 7 `ansible redis -m shell -a "tail -50 /var/log/redis/sentinel.log"` 로 `+sdown` 과 `+odown` 확인

3단계의 조회 결과가 프로세스 정지와 노드 정지를 가릅니다.

`redis_up` 이 0인데 두 target 이 `UP` 이면 프로세스 정지이고, 두 target 이 함께 `DOWN` 이면 노드 정지입니다.

# 감별에서 보는 출력

```
# master 에서
role:master
connected_slaves:1
slave0:ip=10.0.1.7,port=6379,
    state=online,offset=125843,lag=1

# 빠진 replica 에서
Could not connect to Redis at
10.0.1.6:6379: Connection refused

# 남아 있는 replica 에서
master_link_status:up
master_sync_in_progress:0
```

- **connected\_slaves:1** : 구성값 2보다 적음. 이탈 확인
- **Connection refused** : 프로세스가 죽은 것. 이 거부가 곧 감별 근거
- 남은 replica 의 **up** : 끊긴 것이 한 대뿐임을 확인
- **master\_link\_status:down** : 프로세스는 살아 있고 경로만 끊긴 갈래에서 나옴. 지금 재현한 정지에서는 볼 수 없음

# 완화와 복구

| 원인         | 완화                                 | 복구                                  |
|------------|------------------------------------|-------------------------------------|
| 프로세스 정지    | <code>./rb-r01.sh --restore</code> | <code>REPLICA0F</code> 설정이 유지됐는지 확인 |
| 노드 정지      | 노드 복구                              | 자동 재접속 확인                           |
| 네트워크 차단    | 경로 복구                              | 자동 재접속 확인                           |
| 전체 재동기화 반복 | 재접속을 잠시 억제                         | <code>repl-backlog-size</code> 상향   |

되돌리기 전에 로그와 지표를 먼저 남깁니다.  
 재기동하면 그 시점의 상태가 사라집니다. `INFO` 출력과 로그 마지막 부분을 파일로 저장한 뒤 조치합니다.

# 복구 검증

- ☐ 먼저 `./rb-r01.sh --restore` 실행
- ☐ 각 replica 에서 `master_link_status` 가 `up`
- ☐ `SENTINEL replicas mymaster` 가 2개 반환
- ☐ 알람이 `Inactive` 로 돌아옴
- ☐ master 에서 `connected_slaves` 가 2
- ☐ 복제 offset 의 차가 0으로 수렴
- ☐ Grafana 그래프가 원래 값으로 복귀
- ☐ `./rb-r01.sh --status` 로 값이 돌아온 것 확인

replica 하나가 빠지는 것은 성능 문제이고, 정족수가 깨지는 것은 가용성 문제입니다.

이 구분이 세 서비스에 공통으로 적용됩니다. 챕터 09의 MongoDB 정족수 상실과 대비해서 보니다.



# rb-r01.sh 가 한 일

```
# 주입: replica 하나의 redis-server 유닛을 정지
ansible <replica> -m systemd -a "name=redis-server state=stopped" --become
# 복구: 같은 유닛을 기동. --restore 가 이것을 함
ansible <replica> -m systemd -a "name=redis-server state=started" --become
```

- **고른 대상:** Sentinel 에 물어 master 가 아닌 노드 하나. master 를 내리면 자동 전환이 일어나 다른 실습이 됨
- **남기는 것:** `~/rb-r01` 에 어느 노드를 내렸는지 기록. `--restore` 가 그 파일을 보고 되돌림
- **건드리지 않은 것:** 설정 파일, 데이터, 네트워크. 프로세스 하나만 정지

진단이 맞았는지 여기서 대조합니다.

`connected_slaves` 가 줄고 노드 지표는 정상이었던 것이 이 조작의 결과입니다. 프로세스만 내렸으므로 VM 은 멀쩡합니다.

# RB-R01 과 RB-0BS 작성

- 1 RB-R01 을 8절 구조로 작성
- 2 3절 「즉시 확인」에는 「RB-R01 감별 절차」의 1단계부터 3단계까지만 넣음
- 3 4절에 다섯 후보와 갈림 기준을 표로 넣음
- 4 5절에 완화와 복구를 명령 그대로 넣음. 판단 여지를 없앴
- 5 6절에 「RB-R01 복구 검증」의 체크 항목을 넣음
- 6 RB-0BS 에 Redis 지표 칸을 추가하고 「넘어갈 런북」에 RB-R01 을 적음

이 시나리오가 나머지 세 건의 기준 형식입니다.

MongoDB · Kafka · 메모리 압박 런북도 같은 순서로 씁니다.

# RB-R01 작성 예시

# RB-R01 복제 이탈

## 1. 증상

connected\_slaves 2 -> 1. 복제 알람 Firing

## 3. 즉시 확인

redis-cli -h <master> INFO replication

## 4. 원인 감별

redis\_up 0 · target UP -> 프로세스 정지

두 target 함께 DOWN -> 노드 정지

master\_link\_status:down -> 네트워크 차단

# 결론: redis-2 프로세스 정지 (redis\_up 0, UP)

## 5. 복구

```
ansible redis-2 -b -m systemd -a \
    "name=redis-server state=started"
```

## 6. 검증 connected\_slaves 2 · 알람 Inactive

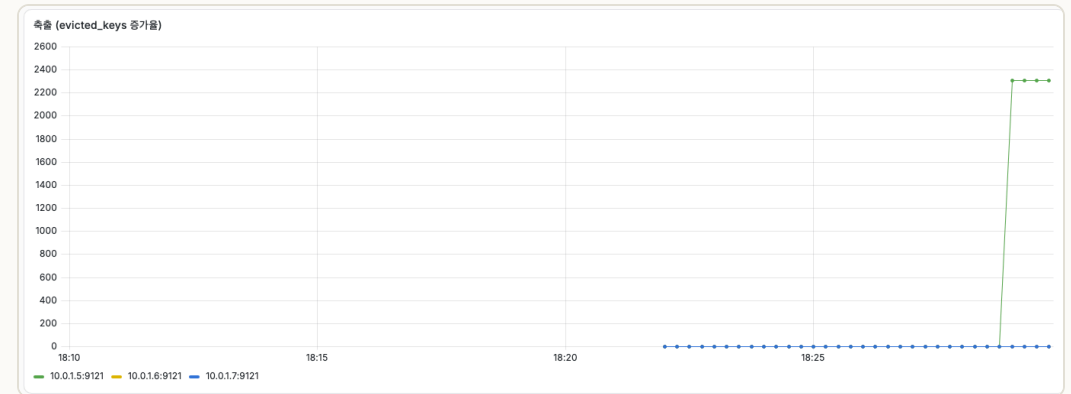
- 3절에 명령만: 왜 그 명령인지는 4절이 맡음
- 4절 끝에 이번 관측과 결론을 적음
- 5절은 그대로 붙일 수 있게 씀

이 형식이 나머지 세 건의 기준입니다.

RB-R02 · RB-M01 · RB-K01 도 같은 구조입니다.

# 장애: 히트율 하락과 쓰기 실패

- 1 ops 노드에서 `cd ~/lab/scenarios`
- 2 `./rb-r02.sh 1` 실행. 상한을 낮추고 적재한 뒤 같은 키 공간을 읽음
- 3 Grafana 의 「축출」, 「히트율」 패널에서 값이 바뀌는 것 확인
- 4 Alerts 화면에서 Redis 메모리 규칙이 Firing 이 되는 것 확인
- 5 VM 메모리 그래프에 변화가 없는 것도 확인
- 6 `./rb-r02.sh 2` 실행. 정책을 `noeviction` 으로 바꾸고 상한을 지금 사용량 아래로 내려 쓰기 거부를 유발



「축출」 패널. evicted\_keys 상승

노드 지표가 정상인데 서비스가 나빠지고 있습니다.

챕터 03의 첫 갈래에서 「노드가 아니다」로 갈린 경우입니다.

# 감별 절차

- 1 ops 노드에서 `redis-cli -h <노드 사설 IP> INFO memory`
- 2 `used_memory` 와 `maxmemory` 의 비 확인
- 3 `maxmemory_policy` 확인. 여기서 eviction 과 쓰기 거부가 갈림
- 4 `INFO stats` 에서 `evicted_keys` 증가율과 히트율 확인
- 5 `mem_fragmentation_ratio` 로 스왑 여부 확인
- 6 노드 메모리 여유와 커널 로그 확인. **서비스 문제인지 노드 문제인지 가름**
- 7 포털의 VM 메모리 그래프와 대조

# eviction 정책에 따른 증상 차이

| 정책                 | 한계 도달 시 동작 | 증상                             |
|--------------------|------------|--------------------------------|
| <b>allkeys-lru</b> | 오래된 키를 축출함 | 쓰기는 되지만 히트율이 떨어짐               |
| <b>noeviction</b>  | 쓰기를 거부함    | <b>OOM command not allowed</b> |

```

> redis-cli -h 10.0.1.5 INFO memory
maxmemory 268435456 # 256 MiB
used_memory 267263384 # 상한에 붙어 있다
maxmemory_policy allkeys-lru

> redis-cli -h 10.0.1.5 INFO stats
evicted_keys 734037 # 73만 키가 뺏겨났다
expired_keys 0 # TTL로 사라진 것은 하나도 없다

# 쓰기는 계속 받아들인다. 대신 오래된 키가 밀려난다.
# evicted_keys 만 오르고 expired_keys 는 0이다. 둘을 세이면 정상 동작을
# 장애로 읽는다.
    
```

**allkeys-lru** 에서 축출하는 중

```

> redis-cli -h 10.0.1.5 INFO memory
used_memory 267921784
maxmemory 241129685 # 사용량보다 낮다
maxmemory_policy noeviction

> redis-cli -h 10.0.1.5 SET probe:room 1
(error) OOM command not allowed when used memory > 'maxmemory'.

# 같은 메모리 부족인데 정책에 따라 증상이 정반대다.
# allkeys-lru에서는 쓰기가 되고 히트율이 떨어진다.
# noeviction에서는 쓰기가 거부된다. 정책을 확인하지 않으면 같은 원인을
# 두 장애로 읽는다.
    
```

**noeviction** 에서 쓰기 거부

같은 메모리 부족인데 증상이 정반대로 보입니다.

한쪽은 「느려졌다」이고 다른 쪽은 「오류가 난다」입니다. 정책을 확인하지 않으면 같은 원인을 두 장애로 읽습니다.

# 완화와 복구

| 원인          | 완화                | 복구                                    |
|-------------|-------------------|---------------------------------------|
| eviction 폭주 | 정책을 상황에 맞게 조정     | <code>maxmemory</code> 재산정, TTL 설계 점검 |
| 쓰기 거부       | 정책을 전환해 쓰기를 우선 수용 | 용량 재산정                                |
| 스왑          | 부하 감소             | 노드 메모리 재산정                            |
| fork 시 급증   | 저장 주기 조정          | <code>maxmemory</code> 를 fork 여유까지 낮춤 |

정책을 바꿔 완화했으면 되돌릴 시점을 함께 정합니다.

`noeviction` 을 `allkeys-lru` 로 바꾸면 쓰기는 다시 받지만 오래된 키가 사라집니다. 그 캐시가 무엇을 담고 있었는지에 따라 판단이 갑니다.

# 복구 검증

- ☐ `evicted_keys` 증가가 멈춤
- ☐ `OOM command not allowed` 가 더 이상 나오지 않음
- ☐ `maxmemory` 가 원래 값으로 돌아옴
- ☐ `./rb-r02.sh --status` 로 두 값을 확인
- ☐ 캐시 히트율이 평상시 값으로 회복
- ☐ `mem_fragmentation_ratio` 가 1과 1.5 사이
- ☐ `maxmemory-policy` 가 `allkeys-lru` 로 돌아옴
- ☐ 알람이 `Inactive` 로 돌아옴

먼저 `./rb-r02.sh --restore` 를 실행합니다.

주입 스크립트는 스스로 되돌리지 않습니다. 이 명령을 실행하지 않으면 `maxmemory` 가 낮은 값으로 남아 이후 실습에서 쓰기가 계속 거부됩니다.



# rb-r02.sh 가 한 일

```
# 1단계: 상한을 8gb 에서 256mb 로 낮추고 400 MB 를 적재
redis-cli CONFIG SET maxmemory 256mb
# 2단계: 정책을 바꾸고 상한을 지금 사용량 아래로 내림
redis-cli CONFIG SET maxmemory-policy noeviction
```

| 단계 | 바꾼 값              | 나온 증상                             |
|----|-------------------|-----------------------------------|
| 1  | maxmemory 를 낮춤    | eviction 이 늘고 히트율이 떨어짐. 쓰기는 계속 성공 |
| 2  | 정책을 noeviction 으로 | OOM command not allowed 로 쓰기 거부   |

**VM 메모리는 처음부터 끝까지 정상이었습니다.**

Redis 자신의 상한을 낮춘 것이지 OS 메모리를 채운 것이 아닙니다. 노드 지표만 보면 아무 일도 없었던 것으로 보입니다.

# RB-R02 와 RB-OBS 작성

- 1 RB-R02 를 8절 구조로 작성
- 2 4절 원인 감별의 첫 갈래를 「정책이 무엇인가」로 둠
- 3 5절에 완화와 복구를 넣되 원상 복구 절차를 명시
- 4 6절에 앞 장의 체크 항목을 넣음
- 5 이미 만든 RB-OBS 의 Redis 줄에 「넘어갈 런북」으로 RB-R02 를 더함
- 6 Redis 의 메모리 지표 이름과 이번에 본 값을 적어 둠

**RB-R02** 는 Redis 전용입니다.

세 서비스가 메모리를 쓰는 방식이 달라 한 문서로 묶이지 않습니다. MongoDB 와 Kafka 는 각자의 지표로 다루고, 6단계에서 적어 둔 이름과 값이 세 서비스를 나란히 놓는 재료가 됩니다.

# RB-R02 작성 예시

# RB-R02 메모리 압박

## 1. 증상

쓰기가 OOM command not allowed 로 실패

## 3. 즉시 확인

redis-cli INFO memory | grep maxmemory

redis-cli INFO stats | grep evicted\_keys

## 4. 원인 감별

noeviction -> 쓰기 거부. 오류가 보임

allkeys-lru -> eviction. 히트율로만 보임

# 관측: evicted 666,433 · 히트율 36.1%

## 5. 복구

redis-cli CONFIG SET maxmemory <원래 값>

redis-cli CONFIG SET maxmemory-policy <원래>

## 6. 검증 쓰기 성공. evicted\_keys 증가 멈춤

- 정책이 증상을 바꿈: 오류로도 히트율 저하로도 나타남
- 원래 값을 먼저 적어 둬: 적어 두지 않으면 되돌릴 수 없음

원래 값을 적어 두지 않고 **CONFIG SET** 을 실행하지 않습니다.

**maxmemory 0** 으로 되돌리면 상한이 사라집니다.

# Redis 리소스 정리

- 1 Redis 구간에서 만든 것이 ops 노드에 있는지 확인. 런북과 적어 둔 값
- 2 `session.tfvars` 의 `stage` 를 4 로 고치고 `terraform apply -var-file=session.tfvars`
- 3 Redis 노드 3대만 사라지고 `ops-1` 이 남는 것 확인
- 4 `session.tfvars` 의 `stage` 를 2 로 고치고 `terraform apply -var-file=session.tfvars`
- 5 MongoDB 노드 3대가 생기는 것 확인
- 6 `terraform output -raw inventory > hosts.ini`
- 7 `scp -i ~/.ssh/rkm hosts.ini azureuser@<ops 공인 IP>:~/ansible/inventory/hosts.ini`

사실 IP 가 겹치므로 `apply` 를 두 번 나눠 합니다.

세 서비스가 사실 IP `10.0.1.5` 부터의 주소를 돌려 씁니다. 한 번에 바꾸면 Redis 가 그 주소를 켜 채로 MongoDB 가 같은 주소를 요청해 Azure 가 거부합니다. `stage 4` 를 한 번 거쳐 갑니다.

`-var stage=2` 로만 주지 않습니다.

`-var` 는 그 실행에만 적용되고 상태에 남지 않습니다. 다음 `apply` 에서 값을 다시 주지 않으면 방금 내린 노드가 되살아납니다.

ops 노드와 공인 IP 는 그대로입니다.

네트워크와 ops 노드는 세션 내내 유지됩니다. ops 노드에 둔 런북과 기록도 남습니다.

# 08

## MongoDB 지표와 알람 규칙

스택 전환

복제 지표

WiredTiger 캐시

포화 지표

규칙 작성

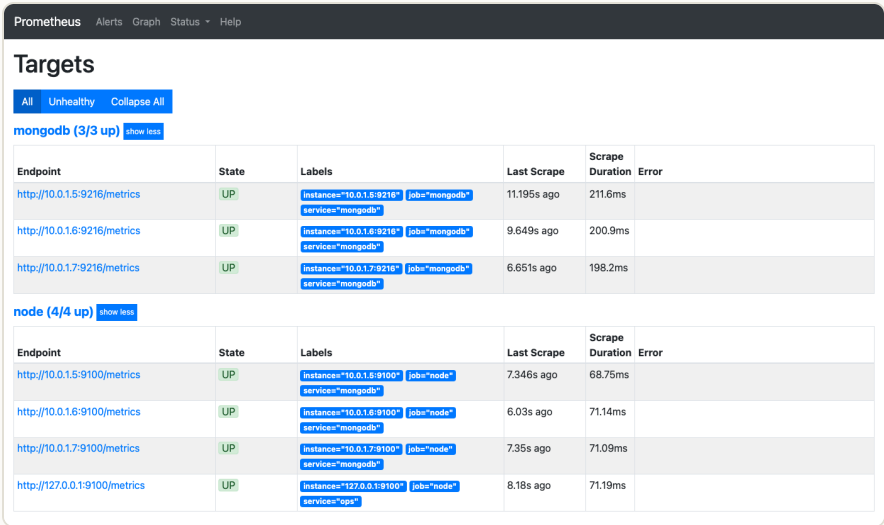
# MongoDB 구성

- 1 앞 장에서 받은 인벤토리가 ops 노드에 덮어써졌는지 확인
- 2 `inventory/hosts.ini` 에 `[mongo]` 와 `[ops]` 두 그룹이 있는 것 확인
- 3 `ansible -m ping all` 로 4대 응답 확인
- 4 `ansible-playbook play-operation.yml` 실행
- 5 마지막 요약에서 실패 노드가 없는 것 확인

**인벤토리에 없는 그룹의 경고는 정상입니다.**

플레이북에 세 서비스의 play 가 모두 들어 있고, 그런 그룹의 play 는 경고 한 줄을 남기고 건너뛹니다. 실패가 아니므로 `failed` 수가 늘지 않습니다.

# MongoDB 전환 확인



Targets 화면. node 4대와 mongodb 3대

| 확인 대상              | 확인 항목                                   |
|--------------------|-----------------------------------------|
| VM 목록              | 4대. ops-1 과 mongo-1 · mongo-2 · mongo-3 |
| 인벤토리               | [mongo] 와 [ops] . 앞 서비스 그룹이 없어야 함       |
| Prometheus Targets | node 4대, mongodb 3대가 UP                 |
| Grafana            | 노드 대시보드와 MongoDB 대시보드                   |
| MongoDB 구성         | 3멤버 Replica Set, Arbiter 없음. 세션 2와 동일   |

앞 서비스 job 이 남아 있으면 인벤토리를 갱신하지 않은 것입니다.  
 없는 노드까지 수집 대상으로 삼아 DOWN 으로 남습니다. 인벤토리를 받아 옮기는 6번과 7번으로 돌아가 인벤토리를 다시 덮어씁니다.

# MongoDB 지표의 출처

- `rs.status()` : 멤버 상태, 각 멤버의 마지막 반영 시각
- `db.serverStatus()` : 캐시, 대기 큐, 연결, 처리량, 지연 분포
- `db.getReplicationInfo()` : oplog 가 담고 있는 시간 범위
- `db.currentOp()` : 지금 도는 작업. **지표가 아니라 목록**
- `mongod.log` : 느린 질의 원문, 선출 기록

`db.currentOp()` 는 그 순간에만 볼 수 있습니다.

오래 도는 질의가 무엇인지는 지표에 남지 않습니다. 장애 중에 조회해야 하고, 끝난 뒤에는 로그의 느린 질의 기록에서만 찾을 수 있습니다.

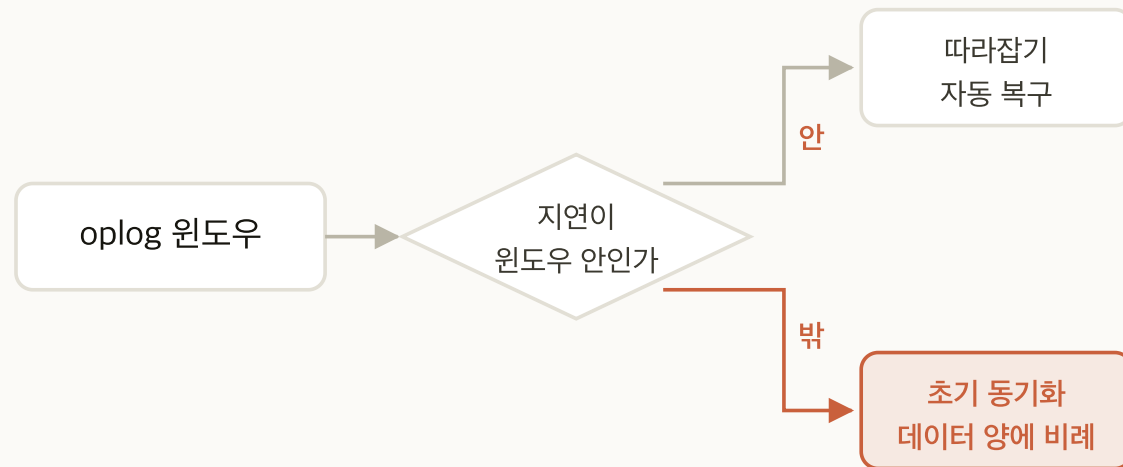


# 복제 지표

| 지표                            | Prometheus 지표                               | 의미               | 판단                           |
|-------------------------------|---------------------------------------------|------------------|------------------------------|
| 멤버별 <code>optimeDate</code> 차 | <code>mongodb_members_optimeDate</code>     | 복제 지연            | oplog 윈도우에 가까워지면 위험          |
| oplog 윈도우                     | 없음. <code>db.getReplicationInfo()</code> 로만 | oplog 가 담은 시간 범위 | 복구 여유의 직접 지표                 |
| <code>health</code>           | <code>mongodb_members_health</code>         | 멤버에 닿는가          | 0이면 그 멤버가 끊긴 것               |
| <code>stateStr</code>         | <code>rs.status()</code> 로만                 | 멤버 역할 문자열        | PRIMARY 1 · SECONDARY 2 가 정상 |
| election 발생 횟수                | 없음. <code>rs.status()</code> 와 로그           | 선출이 몇 번 일어났는가    | 반복되면 네트워크나 시계 문제             |

지연 자체보다 oplog 윈도우와의 거리가 중요합니다.  
 지연 30초는 윈도우가 24시간이면 여유롭고 윈도우가 1분이면 이미 위험합니다. 두 값을 함께 봐야 판단이 됩니다.

# oplog 윈도우와 복구 수단의 관계



- **oplog**: PRIMARY 가 한 변경을 순서대로 담아 두는 고정 크기 기록
- **SECONDARY 가 하는 일**: 그 기록을 읽어 따라감
- **윈도우 안이면**: 밀린 만큼만 읽으면 따라잡음
- **윈도우 밖이면**: 필요한 기록이 이미 지워짐. **처음부터 다시 받아야 함**

정지 시간이 oplog 윈도우를 넘으면 초기 동기화를 다시 합니다.

데이터 양에 비례해 시간이 걸리고 그동안 그 멤버는 정족수에 들어가지 않습니다. 완화 단계에서 이 판단 근거를 먼저 확인합니다.

# WiredTiger 캐시 지표

| 지표                       | 확인 위치                                         | 의미                | 판단                        |
|--------------------------|-----------------------------------------------|-------------------|---------------------------|
| cache used 비율            | wiredTiger.cache ·<br>mongodb_ss_wt_cache_ 계열 | 캐시 사용률            | 80%가 압박 신호                |
| dirty 비율                 | 같은 곳                                          | 아직 디스크에 안 내려간 비율  | 20%가 압박 신호                |
| 애플리케이션 스레드가 수행한 eviction | 같은 곳                                          | 애플리케이션 스레드가 대신 수행 | 0이 아니면 이미 심각              |
| 캐시로 읽어 들인 바이트 증가율        | 같은 곳                                          | 디스크에서 캐시로 올라온 양   | 크면 working set 이 캐시를 넘은 것 |

애플리케이션 스레드 eviction 이 가장 확실한 신호입니다.

평소에는 전용 스레드가 eviction 을 처리합니다. 전용 스레드가 따라가지 못해 **질의**를 처리하던 스레드가 **대신하기 시작하면** 그 질의는 그만큼 늦어집니다. 사용자가 체감하는 순간입니다.

# 포화 지표

| 지표                                            | 의미              | 판단                         |
|-----------------------------------------------|-----------------|----------------------------|
| <code>queues.execution.read.available</code>  | 읽기 동시 실행 슬롯 잔량  | 0이면 포화                     |
| <code>queues.execution.write.available</code> | 쓰기 동시 실행 슬롯 잔량  | 0이면 포화                     |
| <code>globalLock.currentQueue</code>          | 대기 큐            | 0이 아니면 이미 포화               |
| <code>connections.available</code>            | 받을 수 있는 남은 연결 수 | 0에 가까우면 신규 연결 실패 임박        |
| <code>connections.totalCreated</code> 증가율     | 새 연결 생성 빈도      | 폭증은 커넥션 풀 미사용이거나 연결이 계속 끊김 |

슬롯이 0이면 처리량이 더 오르지 않습니다.

MongoDB 는 동시에 실행할 읽기와 쓰기의 수에 상한이 있습니다. 그 상한에 붙으면 요청이 늘어도 처리량은 그대로이고 대기 큐만 자랍니다. **USE 의 Saturation** 이 이것입니다.

슬롯 고갈은 원인이 아니라 결과인 경우가 많습니다. 인덱스 없는 질의가 슬롯을 오래 쥐고 있으면 슬롯이 돌아오지 않습니다. 상한은 같은 곳의 `totalTickets` 이고 2 vCPU 노드에서 읽기·쓰기 각각 4 입니다. `usesThroughputProbing` 이 켜져 있어 고정값이 아닙니다.

# 질의 효율 지표

| 지표                        | 확인 위치                                                 | 의미              | 판단              |
|---------------------------|-------------------------------------------------------|-----------------|-----------------|
| scanned 대 returned 비      | <code>db.serverStatus()</code> 의 <code>metrics</code> | 검사한 문서 대비 반환    | 크면 인덱스가 없거나 잘못됨 |
| <code>scanAndOrder</code> | <code>metrics.operation</code>                        | 인덱스로 정렬하지 못한 횟수 | 정렬용 인덱스가 없는 것   |
| slow query 기록             | <code>mongod.log</code> 와 profiler                    | 느린 질의 원문        | 기본 임계값은 100ms   |
| <code>opLatencies</code>  | <code>db.serverStatus()</code>                        | 읽기·쓰기·명령 지연 분포  | 평균이 아니라 분포      |
| <code>opcounters</code>   | <code>db.serverStatus()</code>                        | 연산 종류별 처리량      | 패턴 변화 감지        |

scanned 대 returned 비가 1에 가까우면 인덱스가 잘 맞는 것입니다.

1000개를 훑어 1개를 돌려주고 있으면 그 질의는 인덱스를 못 쓰고 있습니다. 포털의 디스크 IOPS 그래프가 답하지 못하는 「왜」가 여기 있습니다.

# MongoDB 알람 규칙 작성

- 1 **복제 지연 규칙:** `mongodb_members_optimeDate` 의 PRIMARY 대비 차이가 임계값을 넘을 때
- 2 대시보드 「복제 지연」 패널의 식을 그대로 사용함. **화면에서 본 그래프와 같은 식**
- 3 임계값은 **oplog 윈도우 대비 비율**로 잡음. 앞서 본 기준을 씀
- 4 `for` 절을 붙여 순간 급등에 울리지 않게 함
- 5 **멤버 상태 규칙:** `mongodb_members_health` 가 0 일 때. 지연 규칙과 짝으로 씀
- 6 두 규칙에 `severity` 와 `runbook` 주석 `RB-M01` 을 붙임
- 7 규칙 파일을 반영하고 `Alerts` 화면에서 `Inactive` 확인

지연 규칙만 걸면 가장 나쁜 상태에서 아무것도 울리지 않습니다.

멤버가 완전히 끊기면 `optimeDate` 가 `1970-01-01` 이 되고 지연 식의 `> 0` 필터가 그 멤버를 제외합니다. 그래서 `health` 를 함께 겁니다.

**Redis 에서 한 것과 같은 순서입니다.**

지표를 배우고 그 지표로 규칙을 씁니다.

# 09

## MongoDB 대표 장애와 재현

대표 장애 11

인덱스 없는 질의

정족수 상실

복제 지연 재현

# MongoDB 대표 장애 (1)

| 증상                               | 판별 근거                                                                                                                     | 대응                      |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------|-------------------------|
| <b>w:majority</b> 쓰기의 응답 시간이 늘어남 | 멤버별 <code>optimeDate</code> 차                                                                                             | 지연 노드의 부하 원인 제거         |
| 특정 시각부터 전체가 느려짐                  | scanned 대 returned 비, <code>COLLSCAN</code> , slow query 로그                                                               | 인덱스 추가. 질의 형태 교정        |
| 성능이 계단식으로 무너짐                    | cache used·dirty 비율, 애플리케이션 스레드 eviction                                                                                  | working set 재검토, 인덱스 정리 |
| 쓰기가 전부 실패하고 PRIMARY 가 없음         | <code>rs.status()</code> 에서 전원 SECONDARY, <code>not primary</code> 오류. 기본 read preference 가 <code>primary</code> 라 읽기도 실패 | 멤버 복구로 정족수 회복           |
| 따라잡기가 안 되고 지연이 계속 벌어짐            | 지연과 oplog 윈도우 대조                                                                                                          | 초기 동기화 재수행              |
| 신규 연결이 실패                        | <code>connections.available</code> 이 0, <code>totalCreated</code> 폭증                                                      | 커넥션 풀 설정 점검             |



# MongoDB 대표 장애 (2)

| 증상                          | 판별 근거                                                   | 대응                                 |
|-----------------------------|---------------------------------------------------------|------------------------------------|
| 처리량이 상한에 붙어 더 오르지 않음        | 동시 실행 슬롯 잔량 0, 대기 큐 증가                                  | <b>원인 질의를 먼저 찾음.</b> 상한을 먼저 올리지 않음 |
| 특정 컬렉션이 멈춘 것처럼 보임           | <code>db.currentOp()</code> 의 <code>secs_running</code> | 해당 작업 종료. 인덱스 빌드는 시간을 확인           |
| PRIMARY 가 계속 바뀜             | election 횟수, 노드 간 시계 오차                                 | 네트워크와 시간 동기화 점검                    |
| 쓰기가 오류가 아니라 <b>응답 없이 대기</b> | 정족수 미달 상태에서 Write Concern 미충족                           | 정족수 회복. 쓰기 요청에 timeout 설정          |
| <code>mongod</code> 가 정지    | 파일 시스템 여유, 로그의 쓰기 실패                                    | 디스크 정리 후 재기동                       |

`w:majority` 무한 대기는 오류로 나타나지 않습니다.

정족수가 깨진 상태에서 과반 확인을 요구하는 쓰기는 실패하지 않고 **기다립니다**. 오류 로그를 찾다가 시간을 보내기 쉬운 자리입니다.

# 인덱스 없는 질의

```
ops-1: mongosh --host 10.0.1.5
> db.lagprobe.countDocuments()
12853500
> db.lagprobe.find({ n: 3000000 }).explain("executionStats")
{
  queryPlanner.winningPlan.stage: COLLSCAN
  executionStats.totalDocsExamined: 12853500
  executionStats.nReturned: 4
  executionStats.executionTimeMillis: 4936
}
# 1285만 건을 훑어 4건을 돌려주었다. scanned 대 returned 가 321만 대 1이다.
# 같은 시각 포털의 디스크 IOPS 가 함께 쾜다.
```

COLLSCAN 과 같은 시각의 디스크 IOPS

- **증상:** 특정 시각부터 전체가 느려짐. CPU 와 디스크 IOPS 가 함께 오름
- **무엇으로 아는가:** `explain("executionStats")` 에 `COLLSCAN` . scanned 대 returned 비가 큼. `mongod.log` 의 느린 질의 기록. 포털의 디스크 IOPS 가 그 시각에 급증
- **대응:** 질의 형태에 맞는 인덱스 추가. 정렬이 있으면 정렬 필드까지 포함

두 계층을 잇는 대표 사례입니다.

포털의 IOPS 그래프는 「디스크가 한계인가」를 답하고, `COLLSCAN` 은 「왜 IOPS 가 치솟는가」를 답합니다. 한쪽만 보면 디스크를 늘리게 됩니다.

# 정족수 상실

```
ops-1 : mongosh --host 10.0.1.3

# SECONDARY 두 대를 정지시킨 뒤

> rs.status().members.forEach(m => print(m.name + " " + m.stateStr))
10.0.1.5:27017 SECONDARY
10.0.1.6:27017 (not reachable/healthy)
10.0.1.7:27017 (not reachable/healthy)

> db.getSiblingDB("lab").t.insertOne({ a: 1 })
MongoServerError(NotWritablePrimary): not primary

# 3멤버에서 정족수는 2다. 두 대가 빠지면 남은 하나가 스스로 PRIMARY 가 되지
# 못하고 SECONDARY 로 내려간다. 쓰기가 전면 거부된다.
```

PRIMARY 부재와 not primary 오류

- **증상:** 쓰기가 전부 실패. 읽기도 기본 설정에서는 함께 실패
- **무엇으로 아는가:** `rs.status()` 에서 `stateStr` 이 전원 SECONDARY. 쓰기에 `not primary` 오류. `w:majority` 쓰기는 오류가 아니라 대기
- **왜 그런가:** 3멤버에서 정족수는 2. 두 대가 빠지면 남은 하나가 스스로 PRIMARY 가 될 수 없음
- **대응:** 빠진 멤버를 복구해 정족수를 회복

세션 2 와 목적이 다릅니다.

세션 2 에서는 과반이 깨졌을 때 쓰기가 어떻게 실패하는지를 봤습니다. 여기서는 그 상태를 **증상만 보고 감별합니다.** 무엇을 정지시켰는지 알려 주지 않습니다.

# 슬롯 고갈과 오래 도는 작업

- **슬롯 고갈의 증상:** 요청이 늘어도 처리량이 안 오름
- **무엇으로 아는가:** 슬롯 잔량 0, `globalLock.currentQueue` 증가
- **대응:** 원인 질의를 먼저 찾음. 상한을 올리는 것은 그다음
- **오래 도는 질의와 인덱스 빌드:** 특정 컬렉션이 멈춘 것처럼 보임.  
`db.currentOp()` 의 `secs_running` 으로 확인하고 해당 작업을 종료



인덱스 빌드를 중간에 끊으면 처음부터 다시 합니다.

`db.currentOp()` 에서 진행률을 먼저 보고 판단합니다.

# 선출 반복과 디스크 고갈

```
ops-1 : ansible mongo -b -a "grep ELECTION /var/log/mongodb/mongod.log"

# mongod.log 는 JSON 이다. c 필드가 ELECTION 인 줄만 뽑았다.
18:10:20.512 id 23980 Responding to vote request
18:10:46.807 id 23980 Responding to vote request
18:10:46.808 id 21450 Election succeeded, assuming primary role
18:11:09.563 id 23980 Responding to vote request
18:11:29.560 id 4764800 Not starting an election, since we are not an
electable single node

# 3분 안에 선출이 세 번 일어났다. 한 번은 정상이고 반복이 신호다.
# 횟수를 세어야 정상과 이상이 갈린다.
```

선출 기록이 반복되는 구간

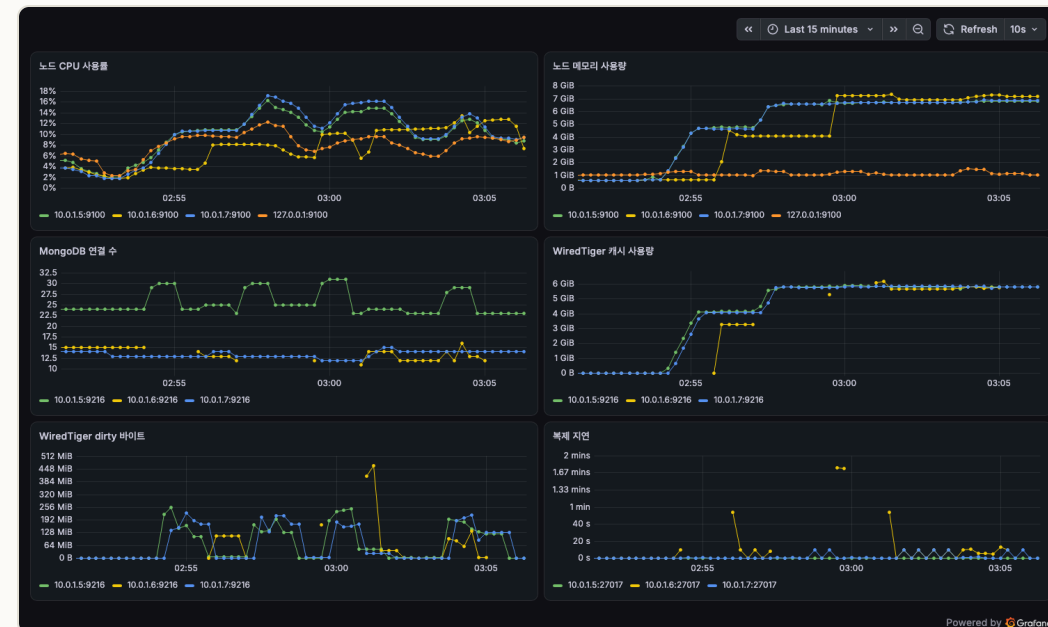
- **선출 반복의 증상:** PRIMARY 가 계속 바뀜. 그때마다 쓰기가 잠깐 끊김
- **무엇으로 아는가:** `mongod.log` 의 `"c":"ELECTION"` 이 반복. 시계 오차 지표
- **대응:** 네트워크 지연과 시간 동기화 점검
- **디스크 공간 고갈:** `mongod` 가 정지. 파일 시스템 여유와 로그의 쓰기 실패 원문으로 확인. 오래된 데이터 정리 후 재기동

선출은 그 자체가 정상 동작입니다.

한 번 일어나는 것은 설계대로 도는 것이고, **반복되는 것이 신호**입니다. 횟수를 세어야 정상과 이상이 갈립니다.

# 장애: 복제 지연 증가

- 1 ops 노드에서 `cd ~/lab/scenarios`
- 2 `./rb-m01.sh` 실행
- 3 Grafana 의 「복제 지연」 패널에서 한 멤버의 지연이 우상향하는 것 확인
- 4 Alerts 화면에서 복제 지연 규칙이 `Firing` 이 되는 것 확인
- 5 이 시각과 두 화면을 적어 둬. **런북 1절의 재료**



「복제 지연」 패널. 한 멤버가 우상향

지연이 「크다」가 아니라 「늘고 있다」가 문제입니다.

일정하게 유지되는 지연은 따라가고 있다는 뜻입니다. 벌어지면 못 따라가는 것입니다.

# 원인 후보 다섯

| 원인                        | 판별 근거                                          | 대응                             |
|---------------------------|------------------------------------------------|--------------------------------|
| SECONDARY 이탈              | <code>rs.status()</code> 에서 해당 멤버가 unreachable | 노드 복구                          |
| SECONDARY 가 oplog 를 못 따라감 | CPU·디스크 IO 상승, 또는 반대로 CPU 가 낮는데 지연만 벌어짐        | 부하 원인 제거. CPU 가 낮으면 프로세스 상태 확인 |
| 대량 쓰기 유입                  | <code>opcounters</code> 급증                     | 유입 속도 조절                       |
| 인덱스 빌드 중                  | <code>db.currentOp()</code> 에 빌드 작업            | 완료를 기다림                        |
| 네트워크 대역 부족                | 노드 지표의 수신량이 상한에 붙음                             | 경로 점검                          |

원인이 SECONDARY 쪽에만 있는 것이 아닙니다.

PRIMARY 에 쓰기가 몰려도 지연이 벌어집니다. 지연이 있는 멤버만 보면 원인을 잘못 지목합니다.

# 감별 절차

- 1 ops 노드에서 `mongosh --host <PRIMARY 사설 IP>`
- 2 `rs.status()` 로 멤버 상태와 각 멤버의 `optimeDate` 확인
- 3 `rs.printSecondaryReplicationInfo()` 로 지연 크기 확인
- 4 `db.getReplicationInfo()` 로 oplog 윈도우 확인
- 5 **지연이 윈도우의 몇 퍼센트인지 계산.** 여기서 복구 수단이 갈림
- 6 지연 노드의 `node_exporter` 지표로 CPU 와 디스크 확인. **낮으면 부하가 아니라 프로세스 쪽**
- 7 `db.currentOp()` 로 그 노드에서 도는 작업 확인
- 8 `opcounters` 로 PRIMARY 의 쓰기 유입량 확인



# 감별에서 보는 출력

```
# 재현 중. 지연이 벌어지는 구간
10.0.1.6:27017 SECONDARY health=1 지연=6초
10.0.1.7:27017 SECONDARY health=1 지연=2초
oplog 윈도우 = 442초

# 재현 뒤. 멤버가 완전히 끊긴 상태
source: 10.0.1.6:27017
{ 'no replication info, yet. State':
  '(not reachable/healthy)' }

# db.getReplicationInfo()
{ logSizeMB: 6413.33, usedMB: 2054.03,
  timeDiff: 940 }
```

- 6초 지연 대 442초 윈도우: 1.4%. 따라잡기가 가능한 구간
- 한 멤버만 지연: PRIMARY 의 쓰기 유입이 아니라 그 노드의 문제
- 끊긴 뒤에는 지연 값이 사라짐: replLag 대신 not reachable

지연은 이탈의 앞 단계입니다.

따라잡지 못하는 멤버는 결국 떨어져 나가고, 그때부터는 지연 값 자체가 나오지 않습니다. health 규칙을 함께 건 이유가 이것입니다.

# 완화와 복구

| 원인            | 완화             | 복구                      |
|---------------|----------------|-------------------------|
| 이탈            | 노드 복구          | 자동 따라잡기 확인              |
| oplog 를 못 따라감 | 그 노드로 가는 읽기 차단 | 부하 원인 제거. 프로세스가 멈췄으면 재개 |
| 대량 쓰기         | 유입 속도 조절       | 용량 재산정                  |
| 인덱스 빌드        | 완료를 기다림        | 빌드 시점을 부하가 적은 때로        |
| 네트워크 대역 부족    | 다른 트래픽 억제      | 경로 점검                   |

윈도우를 넘긴 뒤에는 완화 수단이 없습니다.

초기 동기화는 데이터 양에 비례해 시간이 걸리고, 그동안 그 멤버는 정족수에 들어가지 않습니다. 넘기기 전에 조치하는 것이 유일한 대응입니다.

# 복구 검증

- ☐ 먼저 `./rb-m01.sh --restore` 실행
- ☐ `stateStr` 이 PRIMARY 1 · SECONDARY 2
- ☐ `w:majority` 쓰기 응답 시간이 정상 범위로 복귀
- ☐ Grafana 지연 그래프가 평상시 값으로 복귀
- ☐ 복제 지연 규칙이 `Inactive` 로 돌아옴
- ☐ 모든 멤버의 `health` 가 1
- ☐ 복제 지연이 0초로 수렴
- ☐ 대기 큐가 0
- ☐ `./rb-m01.sh --status` 로 지연이 내려온 것 확인

**지연이 0으로 수렴하는 것까지 확인합니다.**

줄어들기 시작한 것만으로는 따라잡았다고 볼 수 없습니다. 유입이 계속되면 다시 벌어집니다.

# rb-m01.sh 가 한 일

```
# SECONDARY 한 대의 mongod 를 9초 멈추고 1초 푸는 것을 되풀이
ansible <SECONDARY> -m shell -a "pkill -STOP -x mongod" --become
ansible <SECONDARY> -m shell -a "pkill -CONT -x mongod" --become
```

- **SIGSTOP** 을 쓰는 이유: 프로세스를 정지시키면 SECONDARY 로 판정돼 세트에서 빠짐. 멈추기만 하면 세트에는 남은 채 oplog 만 못 읽어 지연이 쌓임
- **9 대 1 인 이유**: 푸는 1초에 하트비트에 답해 무응답 판정을 피함. 4초 멈추고 1초 푸는 값으로는 거의 따라잡아 지연이 6초에서 멈춤
- **그동안 PRIMARY 에 쓰기를 계속 미는 이유**: 쓰기가 없으면 밀릴 것이 없어 지연이 쌓이지 않음

지연이 우상향한 것은 「멈춘 시간이 푸는 시간보다 길어서」입니다.

`--restore` 는 `SIGCONT` 를 보내고 쓰기 루프를 정리합니다. 재개하면 밀린 oplog 를 재생해 따라잡습니다.

# RB-M01 과 RB-OBS 작성

- 1 RB-M01 을 8절 구조로 작성
- 2 3절 「즉시 확인」에 「RB-M01 복제 지연 증가」의 1단계부터 5단계까지를 넣음
- 3 4절에 다섯 후보와 윈도우 대비 지연 비율의 갈림 기준을 넣음
- 4 5절에 완화와 복구를 넣되 윈도우 초과 분기에 경고를 붙임
- 5 6절에 「RB-M01 예상 출력」의 체크 항목을 넣음
- 6 RB-OBS 에 MongoDB 지표 칸을 추가하고 「넘어갈 런북」에 RB-M01 을 적음
- 7 MongoDB 의 메모리 지표 이름과 이번에 본 값을 적어 둬.

**갈림 기준을 숫자로 적습니다.**

「윈도우에 근접하면」이 아니라 「지연이 윈도우의 절반을 넘으면」처럼 적어야 새벽에 판단이 필요 없습니다.

# RB-M01 작성 예시

# RB-M01 복제 지연

1. 증상 한 멤버의 복제 지연이 계속 증가

3. 즉시 확인

```
mongosh --host <PRIMARY>
rs.status() · rs.printSecondaryReplicationInfo()
db.getReplicationInfo()
```

4. 원인 감별

한 멤버만 지연 -> 그 노드의 문제

전 멤버 지연 -> PRIMARY 쓰기 유입

# 관측: replLag 6s / 윈도우 442s = 1.4%

# 관측: CPU 낮음. currentOp 이 간헐 무응답

# 결론: mongo-2 의 mongod 가 멈춰 oplog 미적용

5. 복구 멈춘 mongod 재개. `pkill -CONT -x mongod`

6. 검증 지연 0 수렴 · 알람 Inactive

- 갈림 기준을 숫자로: 「근접하면」이 아니라 「윈도우의 절반」
- 4절 끝에 이번 관측과 결론을 적음

윈도우를 넘기면 따라잡기가 불가능합니다.

5절에 초기 동기화 분기를 반드시 적습니다.

# MongoDB 리소스 정리

- 1 MongoDB 구간에서 만든 것이 ops 노드에 있는지 확인. 런북과 적어 둔 값
- 2 `session.tfvars` 의 `stage` 를 4 로 고치고 `terraform apply -var-file=session.tfvars`
- 3 MongoDB 노드 3대만 사라지고 `ops-1` 이 남는 것 확인
- 4 `session.tfvars` 의 `stage` 를 3 으로 고치고 `terraform apply -var-file=session.tfvars`
- 5 Kafka 노드 3대가 생기는 것 확인
- 6 `terraform output -raw inventory > hosts.ini`
- 7 `scp -i ~/.ssh/rkm hosts.ini azureuser@<ops 공인 IP>:~/ansible/inventory/hosts.ini`

사실 IP 가 겹치므로 `apply` 를 두 번 나눠 합니다.

세 서비스가 사실 IP `10.0.1.5` 부터의 주소를 돌려 씁니다. 한 번에 바꾸면 MongoDB 가 그 주소를 쫓 채로 Kafka 가 같은 주소를 요청해 Azure 가 거부합니다. `stage 4` 를 한 번 거쳐 갑니다.

`-var stage=3` 으로만 주지 않습니다.

`-var` 는 그 실행에만 적용되고 상태에 남지 않습니다. 다음 `apply` 에서 값을 다시 주지 않으면 방금 내린 노드가 되살아납니다.

ops 노드와 공인 IP 는 그대로입니다.

네트워크와 ops 노드는 세션 내내 유지됩니다. ops 노드에 둔 런북 과 기록도 남습니다.

# 10

## Kafka 지표와 알람 규칙

스택 전환

JMX

복제 지표

디스크

규칙 작성



# Kafka 구성

- 1 앞 장에서 받은 인벤토리가 ops 노드에 덮어써졌는지 확인
- 2 `inventory/hosts.ini` 에 `[kafka]` 와 `[ops]` 두 그룹이 있는 것 확인
- 3 `ansible -m ping all` 로 4대 응답 확인
- 4 `ansible-playbook play-operation.yml` 실행
- 5 마지막 요약에서 실패 노드가 없는 것 확인

**Broker 는 한 대씩 올라갑니다.**

플레이북이 `serial: 1` 로 돌립니다. 세 대를 동시에 재시작하면 controller quorum 의 과반 2를 못 채워 Topic 생성과 leader 교체가 멈춥니다.

# Kafka 전환 확인

Prometheus
Alerts
Graph
Status
Help

Targets

All
Unhealthy
Collapse All

kafka (3/3 up)
show less

| Endpoint                     | State | Labels                                               | Last Scrape | Scrape Duration | Error |
|------------------------------|-------|------------------------------------------------------|-------------|-----------------|-------|
| http://10.0.1.5:9308/metrics | UP    | instance="10.0.1.5:9308" job="kafka" service="kafka" | 1.137s ago  | 5.733ms         |       |
| http://10.0.1.6:9308/metrics | UP    | instance="10.0.1.6:9308" job="kafka" service="kafka" | 14.019s ago | 15.68ms         |       |
| http://10.0.1.7:9308/metrics | UP    | instance="10.0.1.7:9308" job="kafka" service="kafka" | 3.656s ago  | 7.044ms         |       |

kafka\_jmx (3/3 up)
show less

| Endpoint                     | State | Labels                                                   | Last Scrape | Scrape Duration | Error |
|------------------------------|-------|----------------------------------------------------------|-------------|-----------------|-------|
| http://10.0.1.5:9404/metrics | UP    | instance="10.0.1.5:9404" job="kafka_jmx" service="kafka" | 4.755s ago  | 499.7ms         |       |
| http://10.0.1.6:9404/metrics | UP    | instance="10.0.1.6:9404" job="kafka_jmx" service="kafka" | 10.412s ago | 435.9ms         |       |
| http://10.0.1.7:9404/metrics | UP    | instance="10.0.1.7:9404" job="kafka_jmx" service="kafka" | 14.069s ago | 365.2ms         |       |

node (4/4 up)
show less

| Endpoint                     | State | Labels                                              | Last Scrape | Scrape Duration | Error |
|------------------------------|-------|-----------------------------------------------------|-------------|-----------------|-------|
| http://10.0.1.5:9100/metrics | UP    | instance="10.0.1.5:9100" job="node" service="kafka" | 2.145s ago  | 70.77ms         |       |
| http://10.0.1.6:9100/metrics | UP    | instance="10.0.1.6:9100" job="node" service="kafka" | 15.018s ago | 78.44ms         |       |
| http://10.0.1.7:9100/metrics | UP    | instance="10.0.1.7:9100" job="node" service="kafka" | 9.656s ago  | 70.27ms         |       |

Targets 화면.
 node 4대와 kafka 3대

| 확인 대상              | 확인 항목                                   |
|--------------------|-----------------------------------------|
| VM 목록              | 4대. ops-1 과 kafka-1 · kafka-2 · kafka-3 |
| 인벤토리               | [kafka] 와 [ops] . 앞 서비스 그룹이 없어야 함       |
| Prometheus Targets | node 4대, kafka 3대, kafka_jmx 3대가 UP     |
| Grafana            | 노드 대시보드와 Kafka 대시보드                     |
| Kafka 구성           | Broker 3대, KRaft combined. 세션 3과 동일     |

앞 서비스 job 이 남아 있으면 인벤토리를 갱신하지 않은 것입니다.

없는 노드까지 수집 대상으로 삼아 DOWN 으로 남습니다. 인벤토리를 받아 옮기는 6번과 7번으로 돌아가 인벤토리를 다시 덮어씹습니다.

# Kafka 지표의 출처

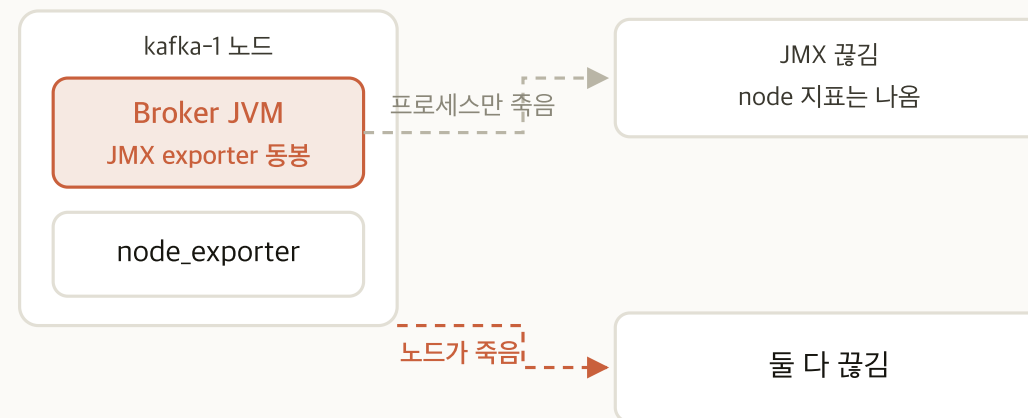
| 출처             | 내는 지표                                  | 노드 정지 시 영향     |
|----------------|----------------------------------------|----------------|
| JMX exporter   | Broker 내부 MBean. 복제·controller·요청 처리   | Broker 와 함께 끊김 |
| kafka_exporter | Topic · Partition · Consumer Group Lag | 그 노드 몫만 끊김     |
| node_exporter  | 디스크 여유, CPU, 메모리, 파일 디스크립터             | 노드가 죽어야 끊김     |
| CLI            | 배치와 상태의 상세                             | 명령을 칠 때만       |

어디서 나오는지가 곧 「그 노드가 죽으면 무엇이 보이지 않는가」입니다.

Broker 가 죽으면 그 Broker 의 JMX 지표는 사라지지만 `node_exporter` 는 남습니다. 챕터 03의 첫 갈래가 Kafka 에서는 이 모양입니다.

# Broker 정지와 지표의 관계

- **JMX exporter**: Broker JVM 안에서 동작. **Broker 와 함께 끊김**
- **node\_exporter**: 별도 프로세스. 노드가 살아 있으면 계속 나옴
- **가려내는 법**: 둘 중 무엇이 끊겼는지로 프로세스 정지와 노드 정지를 나눔



지표가 사라진 것 자체가 신호입니다.

Broker 가 죽었는데 JMX 지표가 계속 나오면 그것이 오히려 이상한 것입니다. 무엇이 끊기고 무엇이 남았는지가 감별의 첫 갈래입니다.

# 복제 지표

| 지표                        | 확인 위치 | 의미                              | 판단                             |
|---------------------------|-------|---------------------------------|--------------------------------|
| UnderReplicatedPartitions | JMX   | ISR 이 RF 에 못 미치는 Partition 수    | 0이 아니면 즉시 확인                   |
| UnderMinIsrPartitionCount | JMX   | ISR 이 min.insync.replicas 미만인 수 | 0이 아니면 acks=all produce 가 거부 됨 |
| OfflinePartitionsCount    | JMX   | leader 가 없는 Partition           | 0이 아니면 완전히 멈춘 것                |
| ActiveControllerCount     | JMX   | 활성 controller 수. Broker 별 값     | 합이 정확히 1                       |
| ISR 축소·확장 횟수              | JMX   | ISR 축소와 확장의 빈도                  | 반복되면 네트워크나 디스크                 |

UnderReplicated 와 UnderMinIsr 는 다릅니다.

앞은 서비스가 돌아가고, 뒤는 acks=all 쓰기가 거부되고 있는 상태입니다.

# 처리와 디스크 지표

| 지표                                                            | 확인 위치                              | 의미                      | 판단                       |
|---------------------------------------------------------------|------------------------------------|-------------------------|--------------------------|
| RequestHandlerAvgIdlePercent                                  | JMX                                | 요청 처리 스레드 유휴율           | 낮으면 Broker 포화            |
| BytesInPerSec · BytesOutPerSec<br>kafka_server_bytes_in_total | JMX                                | 처리량. 카운터라 rate() 가 필요함  | Lag 감별에서 Producer 급증을 배제 |
| 데이터 디스크 여유와 flush 지연                                          | node_exporter · JMX                | log dir 이 있는 파일 시스템     | Kafka 장애 원인 1순위          |
| 열린 파일 디스크립터                                                   | node_exporter                      | 열린 파일 수                 | segment 마다 파일을 여는 구조     |
| JVM 힙 사용률                                                     | jvm_memory_used_bytes{area="heap"} | Broker JVM 의 힙          | 힙을 키우면 page cache 가 줄어듦  |
| Consumer Group 별 Lag                                          | kafka_exporter                     | LEO 에서 커밋된 Offset 을 뺀 값 | 챕터 11 실습의 주역             |

Kafka 장애 원인 1순위가 Kafka 지표에 없습니다. 디스크가 차면 로그 디렉터리가 offline 되는데 그 신호는 node\_exporter 에만 있습니다.

# 대시보드에 없는 지표

| 지표                         | 지표의 의미                                     | 대체 확인 수단                                                         |
|----------------------------|--------------------------------------------|------------------------------------------------------------------|
| unclean leader election 횟수 | 동기화 안 된 replica 가 leader 가 됨. <b>유실 발생</b> | server.log 의 leader 교체 기록과<br>unclean.leader.election.enable 설정값 |
| Broker 별 leader 수          | leader 가 한쪽에 몰림                            | kafka-topics.sh --describe 의 Leader 열                            |
| Consumer 쪽 Rebalance 빈도    | Rebalance 가 반복됨                            | <b>Consumer 로그</b>                                               |

Rebalance 반복은 로그로만 봅니다.

Consumer JVM 에 별도 agent 를 붙이지 않는 한 지표로 나오지 않습니다. 그래서 이 챕터의 Lag 감별에서 로그를 읽는 단계가 들어갑니다.

# JMX 지표 확인

Prometheus 에서 각각 조회합니다

```
kafka_server_under_replicated_partitions
kafka_server_under_min_isr_partition_count
kafka_controller_offline_partitions_count
sum(kafka_controller_active_controller_count)
```

- 1 위 네 지표를 각각 조회
- 2 앞 셋은 Broker 별로 0, 마지막은 **합이 정확히 1** 인 것 확인
- 3 `kafka_exporter` 가 내는 Lag 지표와 이름 규칙이 다른 것 확인

**ActiveControllerCount** 는 Broker 별로 나옵니다.

활성 controller 하나가 1을 내고 나머지 둘이 0을 냅니다. 「정확히 1」은 세 값을 합해서 내리는 판정입니다. 합이 0이면 메타데이터 변경이 전부 멈춰 Topic 생성도 leader 교체도 되지 않습니다.



# Topic 과 Consumer Group 생성

- 1 `kafka-topics.sh --bootstrap-server <Broker 사실 IP>:9092 --create --topic lab.events --partitions 3 --replication-factor 3 --config min.insync.replicas=2`
- 2 `--describe --topic lab.events` 로 **Partition 3개와 ISR 3개** 확인
- 3 `kafka-console-producer.sh` 로 메시지 몇 건 입력 후 종료
- 4 `kafka-console-consumer.sh --group lab-cg --from-beginning` 실행 후 종료
- 5 `kafka-consumer-groups.sh --describe --group lab-cg` 로 **LAG 열의 정상시 값** 확인

`--replication-factor 3` 을 빼면 Broker 한 대가 멈출 때 그 Partition 이 통째로 멈춥니다.

Broker 가 3대이므로 3으로 둡니다. `min.insync.replicas=2` 는 셋 중 둘이 살아 있어야 `acks=all` 쓰기를 받는다는 뜻입니다.

여기서 본 **LAG** 값이 다음 장 임계값의 근거입니다.

Lag 은 정상 운영에서도 0이 아닙니다. 지금 값을 적어 두어야 임계 값을 정할 수 있습니다.

# Kafka 알람 규칙 작성

- 1 **Consumer Lag 규칙:** `kafka_consumergroup_lag` 합이 임계값을 넘을 때
- 2 임계값은 **평상시 처리량으로 따라잡을 수 있는 양**을 기준으로 잡음
- 3 `for` 절을 붙여 순간 적체와 지속 적체를 가름
- 4 `severity` label 을 붙이고 `runbook` 주석에 `RB-K01` 을 적음
- 5 규칙 파일을 반영하고 `Alerts` 화면에서 `Inactive` 확인

**Lag 은 정상 운영에서도 0이 아닙니다.**

앞서 본 평상시 값을 기준으로 잡습니다. 0을 임계로 두면 항상 발동합니다.

**세 서비스 모두 같은 자리에서 규칙을 씁니다.**

지표를 배운 직후입니다. 임계값은 평상시 값 위에서 잡습니다.

# 11

## Kafka 대표 장애와 재현

대표 장애 11

Lag 원인 여섯

분포 기반 분류

Lag 재현

# Kafka 대표 장애 (1)

| 증상                 | 판별 근거                                                                         | 대응                                          |
|--------------------|-------------------------------------------------------------------------------|---------------------------------------------|
| Lag 이 계속 늘고 줄지 않음  | Partition 별 Lag 분포, <code>CURRENT-OFFSET</code> 전진 여부                         | 원인 여섯 갈래로 감별 후 조치                           |
| Consumer 가 계속 재할당됨 | Consumer 로그의 Rebalance 반복                                                     | <code>max.poll.interval.ms</code> 조정, 배치 축소 |
| Broker 가 사실상 정지    | 데이터 디스크 여유, 로그의 log dir 실패                                                    | 오래된 데이터 정리, retention 재산정                   |
| replica 가 모자람      | <code>UnderReplicatedPartitions</code> 가 0 초과                                 | 빠진 Broker 복구                                |
| Producer 가 오류를 받음  | <code>UnderMinIsrPartitionCount</code> 가 0 초과, <code>NotEnoughReplicas</code> | ISR 회복. 임시로 <code>acks</code> 완화            |
| 읽기가 느려짐            | 힙 사용률은 낮는데 디스크 읽기가 늘어남                                                        | 힙을 줄임. 늘리지 않음                               |

# Kafka 대표 장애 (2)

| 증상                           | 판별 근거                                                                                       | 대응                           |
|------------------------------|---------------------------------------------------------------------------------------------|------------------------------|
| Broker 한 대만 바쁨               | <code>--describe</code> 의 <code>Leader</code> 편중, <code>RequestHandlerAvgIdlePercent</code> | preferred leader 로 되돌림       |
| Topic 생성과 leader 교체가 전부 멈춤   | <code>ActiveControllerCount</code> 가 0                                                      | controller 정족수 회복            |
| 특정 Partition 이 완전히 멈춤        | <code>OfflinePartitionsCount</code> 가 0 초과                                                  | 그 Partition 의 replica 복구     |
| 메시지가 사라짐                     | <code>server.log</code> 의 leader 교체 기록과 <code>unclean.leader.election.enable</code> 설정값     | ISR 회복을 기다림. 설정은 끈 채로 둠      |
| Consumer 가 읽으려는 Offset 이 사라짐 | <code>OffsetOutOfRange</code> , 로그 시작 Offset 의 점프                                           | retention 을 Consumer 지연보다 길게 |

**unclean leader election 을 켜면 유실이 일어납니다.**

동기화가 안 된 replica 를 leader 로 올려 가용성을 얻는 대신 그 차이만큼의 메시지를 버립니다. 기본값이 꺼져 있고, 이 과정에서 켜지 않습니다.

# 디스크 고갈과 log dir offline

```
kafka-3 : /opt/kafka/logs/server.log
> ansible kafka -m shell -a "df -h /var/lib/kafka"
/dev/sdc 256G 256G 64M 100% /var/lib/kafka

> Prometheus
node_filesystem_avail_bytes(mountpoint="/var/lib/kafka") 913408
sum(kafka_server_under_replicated_partitions) 53

> tail /opt/kafka/logs/server.log
WARN [ReplicaManager brokers=3] Stopping serving replicas in dir
/var/lib/kafka because the log directory has failed.
KafkaStorageException: The log dir /var/lib/kafka is already offline
due to a previous IO exception.
WARN [ReplicaManager brokers=3] Broker 3 stopped fetcher for partitions
lab.events-0, lab.events-1, lab.events-2, __consumer_offsets-*
ERROR Shutdown broker because all log dirs in /var/lib/kafka have failed

# Kafka 지표에는 '복제본이 모자라다'까지만 나온다. 왜 그런지는 노드의
# 디스크 여유와 server.log 예만 있다. 두 계층을 함께 봐야 하는 자리다.
```

log dir 실패 원문과 디스크 여유

- **증상:** 그 Broker 가 요청을 받지 못함. ISR 에서 빠짐
- **무엇으로 아는가:** `ansible kafka -m shell -a "df -h /var/lib/kafka"` . `server.log` 에 로그 디렉터리 실패 원문. `UnderReplicatedPartitions` 가 동시에 오름
- **왜 그런가:** 로그 디렉터리에 쓰지 못하면 그 Broker 는 자기 몫의 Partition 을 못 맡음
- **대응:** retention 을 줄여 오래된 segment 를 지움. 근본은 용량 재산정

Kafka 지표만 보면 원인이 보이지 않습니다.

`UnderReplicatedPartitions` 가 올랐다는 것만 보고 Broker 를 재기동하면 다시 같은 상태가 됩니다. **디스크 여유를 함께 봐야 원인이 나옵니다.**

# min.insync.replicas 미달

```
ops-1: min.insync.replicas=3, 3broker 만 데 읽지

> kafka-topics.sh --describe --topic lab.events
Partition: 0   Leader: 1   Replicas: 3,1,2   Isr: 1,2   Elr: 3
Partition: 1   Leader: 1   Replicas: 1,2,3   Isr: 1,2   Elr: 3
Partition: 2   Leader: 2   Replicas: 2,3,1   Isr: 2,1   Elr: 3

> kafka-console-producer.sh --producer-property acks=all
ERROR [Producer] Error when sending message
org.apache.kafka.common.errors.NotEnoughReplicasException
NOT_ENOUGH_REPLICAS

> Prometheus
sum(kafka_server_under_min_isr_partition_count) 3
sum(kafka_server_under_replicated_partitions) 53

# Isr 이 2인데 min.insync.replicas 가 3이라 acks=all 쓰기가 아예 받아지지 않는다.
# UnderReplicated 는 53이고 UnderMinIsr 은 3이다. 두 지표의 등급이 다른 이유다.
```

NotEnoughReplicas 와 UnderMinIsr 그래프

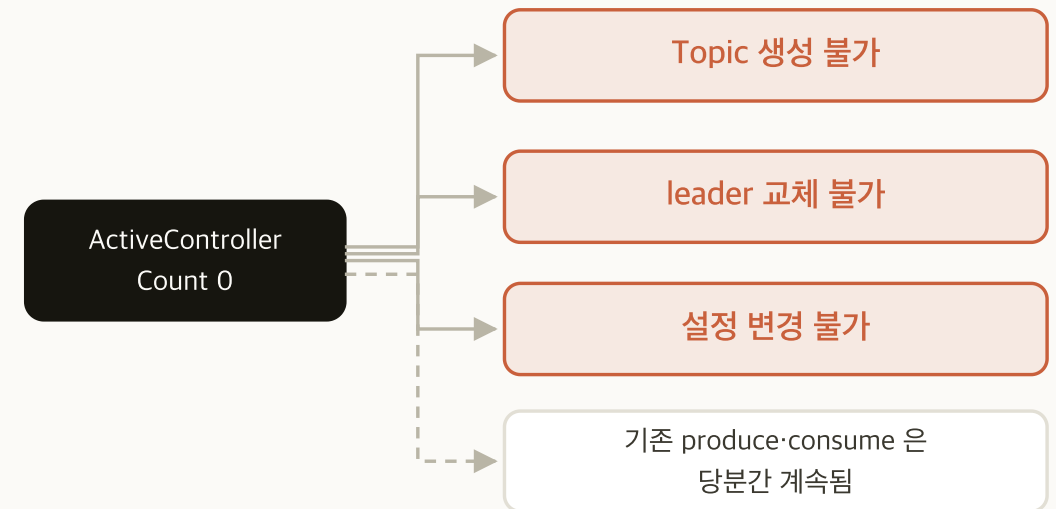
- **증상:** `acks=all` 로 보내는 Producer 가 오류를 받음
- **무엇으로 아는가:** `UnderMinIsrPartitionCount` 가 0 초과. Producer 쪽에 `NotEnoughReplicas`. `--describe` 의 `Isr` 가 설정값보다 짧음
- **왜 그런가:** `acks=all` 은 ISR 전부의 확인을 기다리는데, ISR 크기가 `min.insync.replicas` 보다 작으면 **요청 자체를 받지 않음**
- **대응:** 빠진 Broker 를 복구해 ISR 을 회복

급한 상황에서 `min.insync.replicas` 를 낮추면 유실 위험이 늘어납니다.

쓰기는 다시 들어오지만 replica 가 모자란 채로 받는 것입니다. 완화 수단으로 쓸 때는 되돌릴 시점을 함께 정합니다.

# controller 와 leader 상실

- **controller quorum 상실의 증상:** Topic 생성, leader 교체, 설정 변경 이 전부 멈춤. `ActiveControllerCount` 가 0
- **확인 명령:** `kafka-metadata-quorum.sh ... describe --status`
- **대응:** controller 정족수를 회복. 3대 중 2대가 살아 있어야 함
- **leader 상실:** 특정 Partition 이 완전히 멈춤.  
`OfflinePartitionsCount` 가 0 초과. 그 Partition 의 replica 가 있는 Broker 를 복구



**controller 가 없어도 당장은 메시지가 흐릅니다.**

이미 정해진 leader 로 produce 와 consume 이 계속됩니다. 그래서 알아채는 것이 늦어지고, 그동안 어떤 장애도 자동 복구되지 않는 상태가 이어집니다.



# Retention 오설정

```
ops-1: retention.ms=10000 으로 낮춘 뒤

# retention 을 낮추기 전
> kafka-get-offsets.sh --topic lab.events --time -2
lab.events:0:0
lab.events:1:0
lab.events:2:0

# retention.ms=10000 으로 낮추고 100초 기다린 뒤
> kafka-get-offsets.sh --topic lab.events --time -2
lab.events:0:770784
lab.events:1:752949
lab.events:2:782609

> kafka-console-consumer.sh --partition 0 --offset 1000 \
  --consumer-property auto.offset.reset=none
org.apache.kafka.clients.consumer.OffsetOutOfRangeException:
Fetch position ... is out of range for partition lab.events-0

# 로그 시작 Offset 이 0에서 77만으로 점프했다. 그 사이 Offset 은 사라졌다.
# auto.offset.reset 이 기본값이면 조용히 처음으로 되감아 오류가 안 보인다.
```

OffsetOutOfRangeException 와 segment 목록의 변화

- **증상:** Consumer 가 읽으려는 Offset 이 이미 지워짐
- **무엇으로 아는가:** Consumer 쪽에 `OffsetOutOfRangeException` . 로그 시작 Offset 이 점프. `--describe` 의 Lag 이 이상해짐
- **왜 그런가:** Consumer 가 밀린 사이에 retention 이 그 구간을 지움
- **대응:** retention 을 Consumer 가 밀릴 수 있는 최대 시간보다 길게 잡음
- **기본 설정에서는 오류가 드러나지 않음:** 기본값 `latest` 라 Consumer 가 로그 끝으로 건너뛴. 그 사이 메시지를 읽지 않은 채 지나감

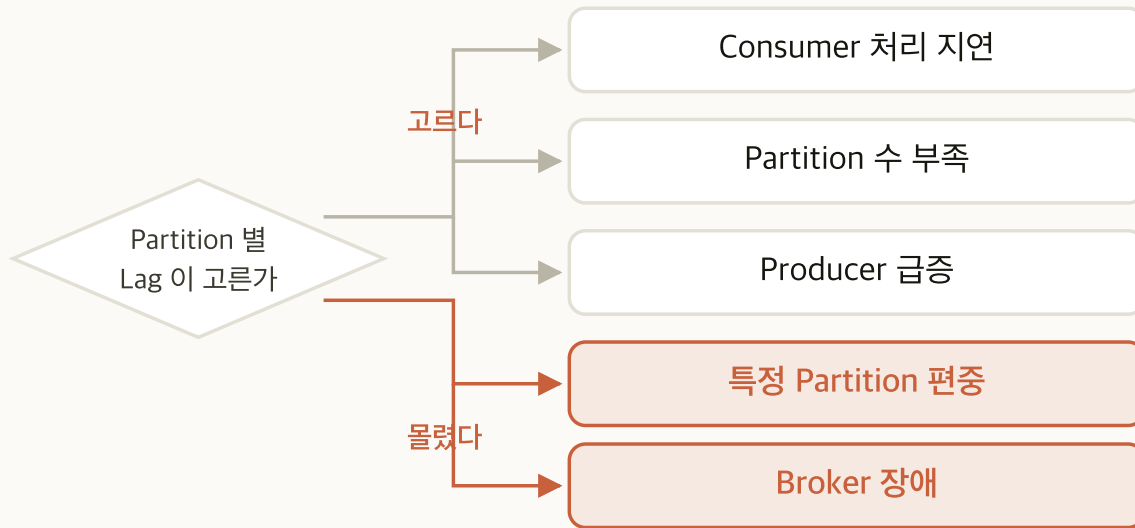
retention 을 줄여 디스크를 확보하는 조치가 이 장애를 만듭니다.

디스크가 차서 retention 을 줄일 때는 Consumer 의 현재 Lag 을 먼저 확인합니다. 두 장애가 서로를 부르는 자리입니다.

# Consumer Lag 의 원인 여섯

| 원인              | 판별 근거                                                                      | 대응                                          |
|-----------------|----------------------------------------------------------------------------|---------------------------------------------|
| Consumer 처리 지연  | Consumer CPU 상승, Lag 이 <b>고르게</b> 증가                                       | 처리 로직 개선, Consumer 증설                       |
| Partition 수 부족  | Consumer 를 늘려도 Lag 이 줄지 않고 일부가 유힬                                          | Partition 증설                                |
| Rebalance 반복    | Consumer 로그의 Rebalance 반복                                                  | <code>max.poll.interval.ms</code> 조정, 배치 축소 |
| Broker 장애       | <code>UnderReplicatedPartitions</code> 증가                                  | Broker 복구                                   |
| Producer 급증     | <code>rate()</code> 를 씌운 <code>BytesInPerSec</code> 급증. Lag 은 <b>결과일 뿐</b> | 용량 증설 또는 유량 제어                              |
| 특정 Partition 편중 | Lag 이 <b>한쪽에만</b> 쌓임                                                       | 키 분포 확인, Partitioner 점검                     |

# Lag 분포로 가르는 첫 갈래



- 한 번의 조회로 여섯이 넷으로: `--describe` 한 번이면 분포가 나옴
- 고르면: Consumer 쪽이거나 유입 쪽
- 몰렸으면: 키 분포거나 특정 Broker
- Rebalance 반복은 별도: 분포는 고르고 로그에서만 갈림

분포를 먼저 보면 불필요한 조치가 줄어듭니다.

Consumer 증설에는 비용과 시간이 듭니다. 명령 한 번으로 후보가 여섯에서 넷으로 줄면 그다음 조치가 정확해집니다.

# Consumer 증설의 한계

| 상황             | Consumer 증설 시 | 이유                                |
|----------------|---------------|-----------------------------------|
| Consumer 처리 지연 | 줄어듦           | 처리 병렬도가 올라감                       |
| Partition 수 부족 | 줄지 않음         | Partition 수가 상한. 남은 Consumer 는 유향 |
| Producer 급증    | 부분적           | 유입 자체가 늘어난 것                      |
| Rebalance 반복   | 오히려 나빠짐       | 멤버가 바뀔 때마다 Rebalance 가 또 일어남      |

Rebalance 반복 상태에서 Consumer 를 늘리면 Lag 이 더 나빠집니다.

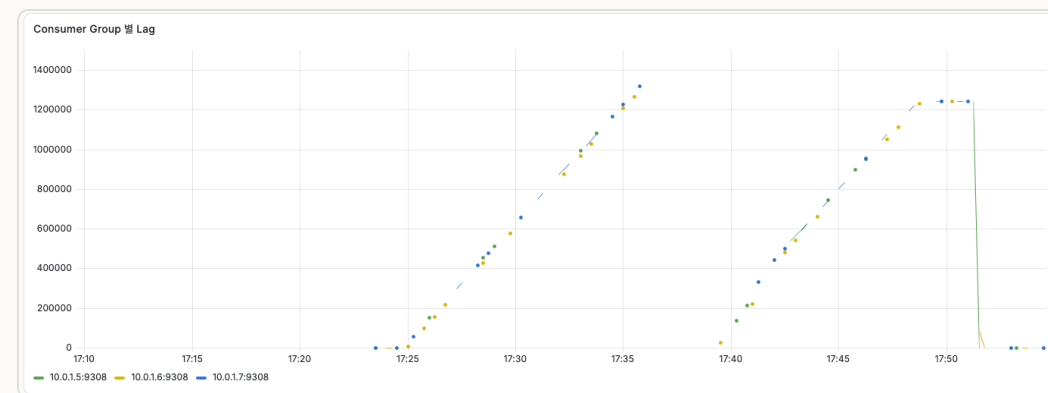
멤버 변경이 Rebalance 를 부르고, Rebalance 중에는 처리가 멈춥니다. 원인을 확정하기 전에 증설부터 하면 안 되는 이유가 이것입니다.

Partition 수는 늘릴 수만 있고 줄일 수 없습니다.

늘리면 기존 키의 Partition 매핑이 깨져 같은 키가 다른 Partition 으로 갑니다. **완화 단계에서 Partition 을 늘리지 않습니다.**

# 장애: Consumer Lag 증가

- 1 ops 노드에서 `cd ~/lab/scenarios`
- 2 `./rb-k01.sh` 실행
- 3 Grafana 의 「Consumer Group 별 Lag」 패널에서 Lag 이 우상향하는 것 확인
- 4 Alerts 화면에서 Lag 규칙이 Firing 이 되는 것 확인
- 5 적어 둔 평상시 LAG 값과 대조



「Consumer Group 별 Lag」 패널. 우상향

Lag 이 늘었다는 것만으로는 원인을 모릅니다.

여섯 갈래 중 무엇인지 앞 세 장의 감별 순서로 좁혀 갑니다.

# 감별 절차

- 1 ops 노드에서 `kafka-consumer-groups.sh --bootstrap-server <IP>:9092 --describe --group <그룹>`
- 2 **Partition 별 Lag 분포를 봄.** 고른 분포인지 편중인지로 1차 분류
- 3 `rate(kafka_server_bytes_in_total[5m])` 그래프로 Producer 급증 여부 배제
- 4 **분포가 몰렸을 때만:** `UnderReplicatedPartitions` 로 Broker 장애 여부 배제
- 5 `CURRENT-OFFSET` 이 전혀 움직이지 않는 Partition 이 있는지 확인
- 6 **Consumer 로그에서 Rebalance 발생 여부 확인**
- 7 남은 후보가 Partition 수 부족이면 Consumer 를 하나 늘려 보고 Lag 변화 관찰

# Lag 분포 확인

| GROUP  | TOPIC      | PARTITION | CURRENT-OFFSET | LOG-END-OFFSET | LAG   |
|--------|------------|-----------|----------------|----------------|-------|
| lab-cg | lab.events | 1         | 230299         | 242009         | 11710 |
| lab-cg | lab.events | 0         | -              | 233869         | -     |
| lab-cg | lab.events | 2         | -              | 235875         | -     |

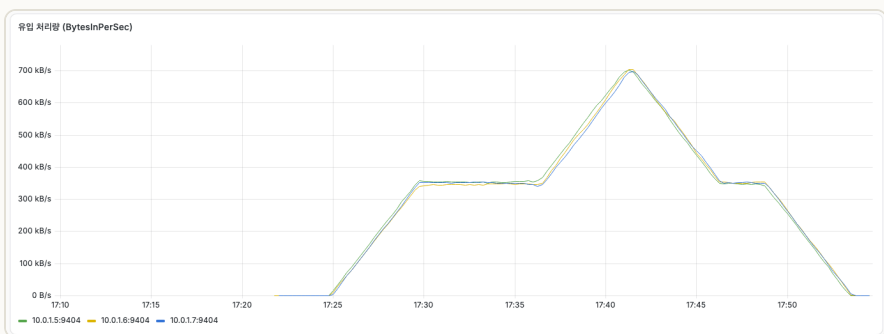
- **CURRENT-OFFSET** 이 **-** 인 Partition: 맡은 Consumer 가 커밋하기 전에 재할당됨
- 세 Partition 의 **LOG-END-OFFSET** 이 비슷함: 유입은 고르게 들어옴. 편중 배제

「-」 표시를 Lag 숫자보다 먼저 봅니다.

커밋된 Offset 이 없는 Partition 은 Lag 을 계산할 수 없습니다. 숫자를 읽기 전에 몇 개가 - 인지부터 봅니다.

kafka-consumer-groups.sh --describe --group lab-cg 의 실제 출력입니다. 남은 후보는 Consumer 처리 지연, Partition 수 부족, Producer 급증, Rebalance 반복 넷입니다. 조회 시점에 따라 표가 비거나 「has no active members」가 나오기도 합니다.

# Producer 급증 배제



rate() 를 씌운 BytesInPerSec

- 유입량이 그대로: Producer 급증 배제
- 남은 후보 셋: Consumer 처리 지연, Partition 수 부족, Rebalance 반복
- 다음 단계: Consumer 로그 확인

**Producer 급증부터 배제해야 원인을 잘못 지목하지 않습니다.**

유입이 두 배가 된 상황이라면 Consumer 는 정상이고 용량 문제입니다. Consumer 를 아무리 손봐도 해결되지 않습니다.



# Consumer 로그에서 확정

```
ops-1: grep -i rebalance /tmp/rkm-consumer-*.log

> grep -hoIE "rebalanc[a-z]*|assign[a-z]*" /tmp/rkm-consumer-*.log | sort | uniq -c
40 assignment
12 rebalance

# 90초 동안 Rebalance 가 12회, 재할당이 40회 일어났다.
# max.poll.interval.ms 를 넘겨 그룹에서 제외되고, 다시 들어오면서 또 일어난다.
#
# 이 값은 지표로 나오지 않는다. Consumer JVM 에 별도 agent 를 붙이지 않기
# 때문이다. 여섯 갈래 중 로그를 봐야만 갈리는 유일한 갈래다.
```

Rebalance 가 반복되는 구간

- **어디서:** `/tmp/rkm-consumer-1.log` 부터 `-3.log` . 주입 스크립트가 Consumer 출력을 남기는 자리
- **찾을 것:** 그룹 재가입과 재할당이 짧은 간격으로 반복되는 기록
- **함께 보이는 것:** poll 간격 초과로 그룹에서 제외됐다는 기록
- **확정: Rebalance 반복**
- **지표로 갈리지 않는 이유:** Consumer JVM 에 별도 agent 를 붙이지 않아 이 값은 Prometheus 에 들어오지 않음

여섯 갈래 중 지표만으로 끝나지 않는 유일한 갈래입니다.

Lag 분포는 고르고 유입량도 정상이라 지표 화면에서는 「Consumer 가 느리다」까지만 보입니다. 왜 느린지는 로그에 있습니다.

# 완화와 복구

| 원인             | 완화             | 복구                                                         |
|----------------|----------------|------------------------------------------------------------|
| Consumer 처리 지연 | Consumer 증설    | 처리 로직 개선                                                   |
| Partition 수 부족 | 즉시 완화 수단 없음    | Partition 증설                                               |
| Rebalance 반복   | 가져오는 건수를 줄임    | <code>max.poll.interval.ms</code> 조정, static membership 적용 |
| Broker 장애      | 남은 Broker 로 버팀 | Broker 복구                                                  |
| Producer 급증    | 유량 제어          | 용량 증설                                                      |

Rebalance 반복의 완화는 배치를 줄이는 것입니다.

한 번에 가져오는 건수를 줄이면 처리 시간이 짧아져 poll 간격 안에 들어옵니다.

완화 단계에서 Partition 을 늘리지 않습니다.

되돌릴 수 없고 기존 키의 매핑이 깨집니다.

# 복구 검증

- ☐ 먼저 `./rb-k01.sh --restore` 실행
- ☐ 설정을 고친 Consumer 를 실행해 쌓인 것을 소진  
`kafka-console-consumer.sh --bootstrap-server <IP>:9092 --topic lab.events --group lab-cg`
- ☐ 모든 Partition 의 `CURRENT-OFFSET` 이 함께 전진
- ☐ Lag 규칙이 `Inactive` 로 돌아옴
- ☐ Lag 이 더 이상 늘지 않음
- ☐ Consumer 로그에 Rebalance 기록이 끊김
- ☐ `UnderReplicatedPartitions` 가 0 유지

주입을 멈추는 것과 Lag 이 사라지는 것은 다릅니다.

`--restore` 는 느린 Consumer 와 Producer 를 멈출 뿐이고, 이미 쌓인 것은 그대로 남아 알람이 계속 울립니다. 정상 Consumer 가 그것을 읽어 내야 `Inactive` 로 돌아옵니다.

# rb-k01.sh 가 한 일

| 조작                                | 값               | 효과                                             |
|-----------------------------------|-----------------|------------------------------------------------|
| Producer                          | 초당 2,000건       | Lag 이 쌓일 유입을 만듦                                |
| Consumer                          | 3개. 한 줄 읽고 2초 쉬 | 처리 속도가 유입을 못 따라감                               |
| <code>max.poll.interval.ms</code> | 20000           | 쉬는 시간이 이 값을 넘으면 그룹에서 빠져 <b>Rebalance</b> 가 반복됨 |

- 여섯 원인 중 무엇이었나: Consumer 처리 지연이 아니라 **Rebalance 반복**. 쉬는 동안 poll 을 못 해 그룹에서 빠지고 다시 들어오는 것이 되풀이 됨
- Consumer 를 늘려도 안 되는 이유: 늘린 Consumer 도 같은 이유로 빠짐. Rebalance 가 더 잦아짐

Lag 분포가 특정 Partition 에 몰리지 않고 전 Partition 에서 함께 오른 것이 단서였습니다.  
한쪽으로 치우치면 키 편향이고, 고르게 오르면 Consumer 쪽 문제입니다.

# Broker 상태 일괄 확인 명령

```
# Topic 배치와 ISR
kafka-topics.sh --bootstrap-server <IP>:9092 \
  --describe --topic <Topic>

# controller quorum 상태
kafka-metadata-quorum.sh \
  --bootstrap-server <IP>:9092 describe --status

# Consumer Group 과 Lag
kafka-consumer-groups.sh --describe \
  --bootstrap-server <IP>:9092 --group <그룹>

# 디스크 여유
ansible kafka -m shell -a "df -h /var/lib/kafka"
```

- **--describe** : Leader · Replicas · Isr 를 함께 보여줌
- **describe --status** : kafka-metadata-quorum.sh 는 위치 인자를 씀
- 네 명령이 런북 3절의 재료: 3분 안에 실행할 수 있는 명령

# RB-K01 과 RB-OBS 작성

- 1 RB-K01 을 8절 구조로 작성
- 2 3절 「즉시 확인」에 「Broker 상태 일괄 확인 명령」의 네 명령을 넣음
- 3 4절에 여섯 갈래와 분포로 가르는 첫 갈래를 넣음
- 4 5절에 완화와 복구를 넣되 Partition 증설에 경고를 붙임
- 5 6절에 「RB-K01 복구 검증」의 체크 항목을 넣음
- 6 RB-OBS 에 Kafka 지표 칸을 추가하고 「넘어갈 런북」에 RB-K01 을 적음
- 7 Kafka 의 메모리 지표 이름과 이번에 본 값을 적어 둬.

4절에 로그를 보는 단계를 반드시 넣습니다.

지표만으로 끝나지 않는 갈래가 있다는 것이 이 런북의 핵심입니다.

# RB-K01 작성 예시

# RB-K01 Consumer Lag

1. 증상 Lag 이 계속 늘고 줄지 않음

3. 즉시 확인

```
kafka-consumer-groups.sh --describe \
    --group lab-cg
```

# Grafana: Lag · bytes\_in · UnderReplicated

4. 원인 감별

분포 편중 -> 키 쏠림 · Broker 장애

분포 고름 -> 처리 지연 · Partition 부족

Producer 급증 · Rebalance 반복

# 관측: 분포 고름 · bytes\_in 평상시

# 결론: Rebalance 반복 (Consumer 로그)

5. 복구 max.poll.interval.ms 상향

6. 검증 Lag 0 수렴 · 알람 Inactive

- 4절에 로그 단계를 반드시: 지표만으로 끝나지 않는 갈래가 있음
- 4절 끝에 이번 관측과 결론을 적음

Partition 증설은 되돌릴 수 없습니다.

줄이는 방법이 없으므로 5절에 그 사실을 함께 적습니다.

# Kafka 리소스 정리

- 1 Kafka 구간에서 만든 것이 ops 노드에 있는지 확인. 런북과 적어 둔 값
- 2 각자 PC 에서 `session.tfvars` 의 `stage` 를 4 로 고침
- 3 `terraform plan -var-file=session.tfvars` 로 Kafka 노드 3대 삭제 확인
- 4 `terraform apply -var-file=session.tfvars`
- 5 `terraform output -raw inventory > hosts.ini`
- 6 `scp -i ~/.ssh/rkm hosts.ini azureuser@<ops 공인 IP>:~/ansible/inventory/hosts.ini`

**-var stage=4** 로만 주지 않습니다.

Terraform 의 `-var` 는 그 실행에만 적용되고 상태에 남지 않습니다. 다음 `apply` 에서 값을 다시 주지 않으면 파일의 값으로 돌아가 방금 내린 노드가 되살아납니다.

**ops 노드와 공인 IP 는 그대로입니다.**

네트워크와 ops 노드는 세션 내내 유지됩니다. ops 노드에 둔 런북과 기록도 남습니다.



# 12

## 메모리 압박 3종과 과정 마무리

메모리 주체

성능 저하 양상

잘못된 대응

메모리 대비

리소스 정리

# 서비스별 메모리 사용 주체



- **Redis**: 데이터 자체가 메모리. 남긴 절반은 fork 여유
- **MongoDB**: WiredTiger 캐시가 성능을 좌우. 나머지는 운영체제 몫
- **Kafka**: 힙은 작고 **page cache** 가 주역

주체가 다르므로 대응도 다릅니다.

같은 「메모리 부족」이라도 무엇을 늘리고 무엇을 줄일지가 서비스마다 반대입니다.

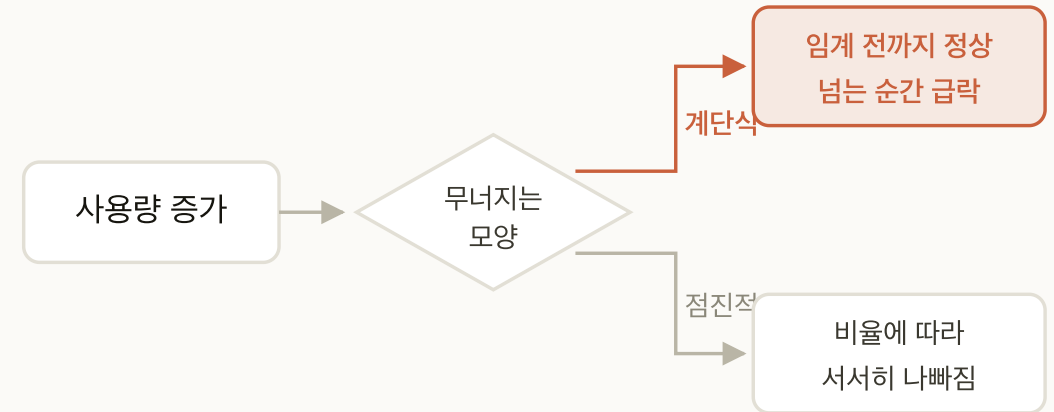
# 메모리 압박 3종 대비

|          | Redis                              | MongoDB             | Kafka                |
|----------|------------------------------------|---------------------|----------------------|
| 부족 시 증상  | eviction 또는 쓰기 거부                  | 디스크 IO 로 강등         | page cache 축출        |
| 성능 저하 양상 | 계단식                                | 계단식                 | 점진적                  |
| 먼저 보는 지표 | 메모리 사용률, <code>evicted_keys</code> | cache used·dirty 비율 | 힙 사용률, 디스크 읽기량       |
| 확정하는 지표  | 히트율 하락                             | 애플리케이션 스레드 eviction | page cache 대비 디스크 읽기 |
| 흔한 오설정   | <code>maxmemory</code> 미설정         | 캐시를 넘는 working set  | 힙을 크게 잡음             |
| 대응       | 정책 조정, 증설                          | 인덱스 정리, 증설          | 힙을 줄임                |
| 잘못된 대응   | <code>maxmemory</code> 를 물리까지 올림   | 캐시를 물리까지 올림         | 힙을 늘림                |

Kafka 에서 힙을 늘리면 그만큼 page cache 가 줄어 읽기가 디스크로 내려갑니다. 다른 두 서비스와 반대로 작동합니다.

# 계단식 하락과 점진적 하락

- 계단식 (Redis · MongoDB): 어느 지점까지 멀쩡하다가 **한 번에 무너짐**
- 무너지는 지점: Redis 는 eviction 이 시작되는 시점, MongoDB 는 working set 이 캐시를 넘는 시점
- 점진적 (Kafka): 디스크 읽기 비율이 서서히 올라감
- 왜 중요한가: 계단식은 「아직 괜찮다」가 위험한 판단



계단식에서는 여유를 미리 확보해야 합니다.

80%에서 90%로 가는 동안 아무 변화가 없다가 91%에서 무너집니다. 지표가 평온하다는 것이 안전하다는 뜻이 아닙니다.

# VM 메모리 그래프의 한계

| 상황                  | VM 메모리 그래프     | 서비스 지표                 |
|---------------------|----------------|------------------------|
| Redis eviction 시작   | 사용률 높음. 평소와 같음 | evicted_keys 가 오름      |
| MongoDB 캐시 초과       | 사용률 높음. 평소와 같음 | 애플리케이션 스레드 eviction 발생 |
| Kafka page cache 축출 | 사용률 높음. 평소와 같음 | 디스크 읽기량이 오름            |

셋 다 VM 메모리 그래프가 똑같이 보입니다.

세 서비스 모두 평소에도 메모리를 거의 다 씁니다. 그것이 정상 동작이므로 그래프만으로는 아무것도 갈리지 않습니다. **챕터 02 관측 계층 구분의 실제 사례**입니다.

# 나머지 두 서비스의 같은 상황

| 서비스     | 현재 확인 항목                                                  | 압박 시의 변화                                  |
|---------|-----------------------------------------------------------|-------------------------------------------|
| MongoDB | <code>mongodb_ss_wt_cache_</code> 계열의 used·dirty 비율, 대기 큐 | used 가 임계값을 넘고 애플리케이션 스레드가 eviction 을 시작함 |
| Kafka   | <code>jvm_memory_used_bytes{area="heap"}</code> , 디스크 읽기량 | 힙은 그대로인데 디스크 읽기만 늘어남                      |

- **재현하지 않는 이유:** MongoDB 는 working set 을 캐시 위로 올려야 하고, Kafka 는 page cache 가 밀릴 만큼 쌓아야 함
- **대신 하는 것:** 각 구간에서 적어 둔 값을 꺼내 표로 나란히 비교

Kafka 에서 힙을 늘리는 것이 대응이 아닙니다.

힙 사용률이 낮는데 읽기가 느리면 page cache 가 밀린 것이고, 힙을 늘리면 더 밀립니다.

Kafka 힙을 줄이려면 롤링 재시작이 필요합니다.

세 Broker 를 동시에 내리면 controller 정족수를 잃습니다. 한 대씩 내립니다.

# 리소스 정리

- 1 ops 노드에 둔 런북 다섯 건을 각자 PC 로 내려받음. `scp -i ~/.ssh/rkm azureuser@<ops 공인 IP>:~/runbook/* .`
- 2 각자 PC 의 `lab/terraform/04-operation` 에서 `terraform destroy -var-file=session.tfvars`
- 3 포털에서 리소스 그룹이 사라진 것 확인

`destroy` 는 남은 `ops-1` 과 디스크·네트워크를 전부 삭제합니다. 되돌릴 수 없습니다.

런북은 `ops-1` 안에만 있습니다. 1단계를 건너뛰면 이들 동안 쓴 것이 함께 사라집니다.

## 2일간의 산출물

| 산출물        | 건수 | 내용                                    |
|------------|----|---------------------------------------|
| 기본 모니터링 런북 | 1  | 무엇을 어디서 보고 정상 범위가 얼마인가                |
| 장애 대응 런북   | 4  | 복제 이탈 · 복제 지연 · Consumer Lag · 메모리 압박 |

자기 환경에 맞게 좁혀 쓰는 것이 남은 일입니다.

이 자료의 장애 목록은 일반적인 것이고, 실제로 쓰는 런북은 자기 환경의 노드 이름과 임계값이 들어가야 합니다.



# 클라우드 비의존 범위

| 단계     | Azure                           | 온프레미스나 다른 클라우드에서               |
|--------|---------------------------------|--------------------------------|
| 인프라 생성 | Terraform                       | 바뀜. 제공자에 맞게                    |
| 구성     | Ansible                         | 그대로                            |
| 관측     | Prometheus · Grafana            | 그대로                            |
| 인프라 지표 | 포털 · <code>node_exporter</code> | <code>node_exporter</code> 만으로 |
| 장애 대응  | 각 서비스의 명령과 로그                   | 그대로                            |
| 런북     | 이 세션에서 쓴 것                      | 그대로                            |

클라우드 의존은 인프라 생성 단계까지였습니다.

그 이후의 구성 · 운영 · 모니터링 · 장애 대응은 어디서든 그대로 통합니다.